Software Defined Radio and GNURadio

Master 1 « Internet of Things »

François Spies, Stéphane Givron, Hussein Zeaiter
Laboratoire FEMTO-ST
UMR CNRS
Université de Franche-Comté
France

Francois.spies@ubfc.fr

Outlines

Before « Software Defined Radio »

Architecture of « Software Defined Radio »

Software « GNURadio »

Generating mathematical signal

Filtering and Modifying signal

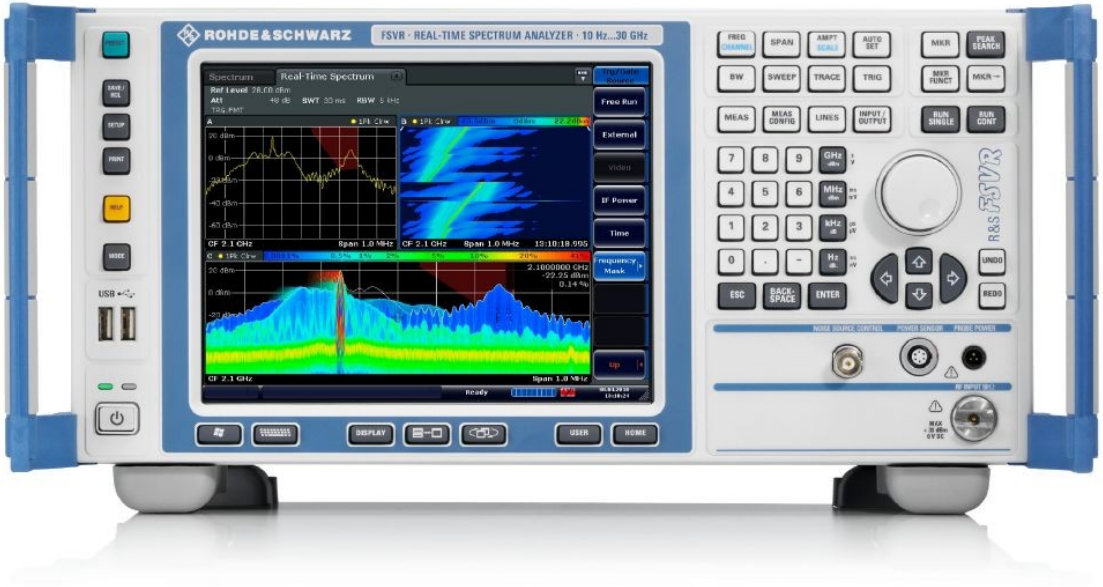
Receiving Analogic Radio Signal

Receiving Digital Radio Signal

Transmitting Radio Signal

Before « Software Defined Radio »

- In order to receive and visualize a radio signal, we used a Spectrum Analyser Box



- Drawbacks
  - Usable for Electronic Engineer
  - Very hard to manipulate for computer scientist point of view
  - Closed environment and costly box at around 40k€ to 60k€

History of « Software Defined Radio »

- In 1970-1980, first works around digital receiver
- In 1990, the term « Software Defined Radio » is used. Works on military and research activities conduct to a few specific hardware
- In 2000, first SDR with « System on Chip » appear on the market
- 2004, Ettus Research present the first SDR model for the market (around 5 to 10k€)
- Around 2010, first DVB-T device on the public market, to receive digital TV used the RTL chipset for around 20 €.

A « Software Defined Radio »

- Around 2016: First model was a « hijacked » use from a DVB-T USB key
- With the transformation of terrestrial TV from analogic to digital transmission, Digital Video Broadcasting - Terrestrial (DVB-T), a new device has been sold to watch TV on computers
- Instead of solding a dedicated chipset with DVB-T demodulation and MPEG decoding, the choice was to propose a minimal hardware for only receiving a radio signal, then adding demodulation and decoding in a piece of software.
- Advantages
  - in case of new modulations or compression format, it is easy to upgrade the software without changing the hardware
  - Low cost at around 15€

femto-st SCIENCES & TECHNOLOGIES

Why « Software Defined Radio » ?

- Previously, the only way to build a receiver was to join a list of electronic devices. For each modification and improvment, it was necessary to rebuild a completly new hardware.
- Using a SDR allow to simply modify a piece of software and then you have a new receiver including new functionnalities.
- Using SDR today, allow to offer a lot of modulation standards. It is easy to improve cellular networks modulation from, 2G to 5G and WiFi from 802.11b to 802.11ax and so on
- When you launch a satellite, it is possible to modify and improve the characteristics of the modulation and coding even 10 years later.

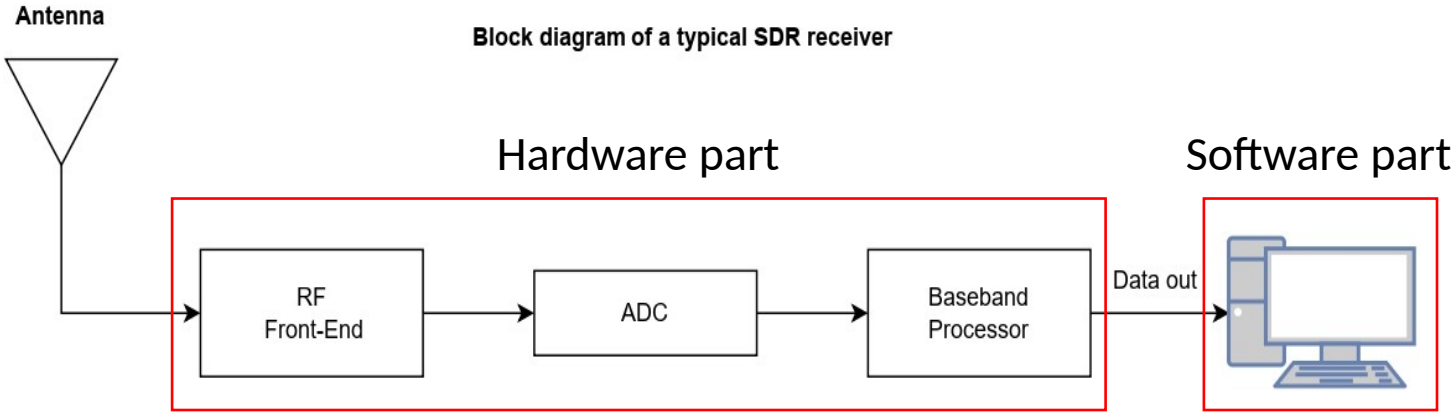
How to build a receiver ?

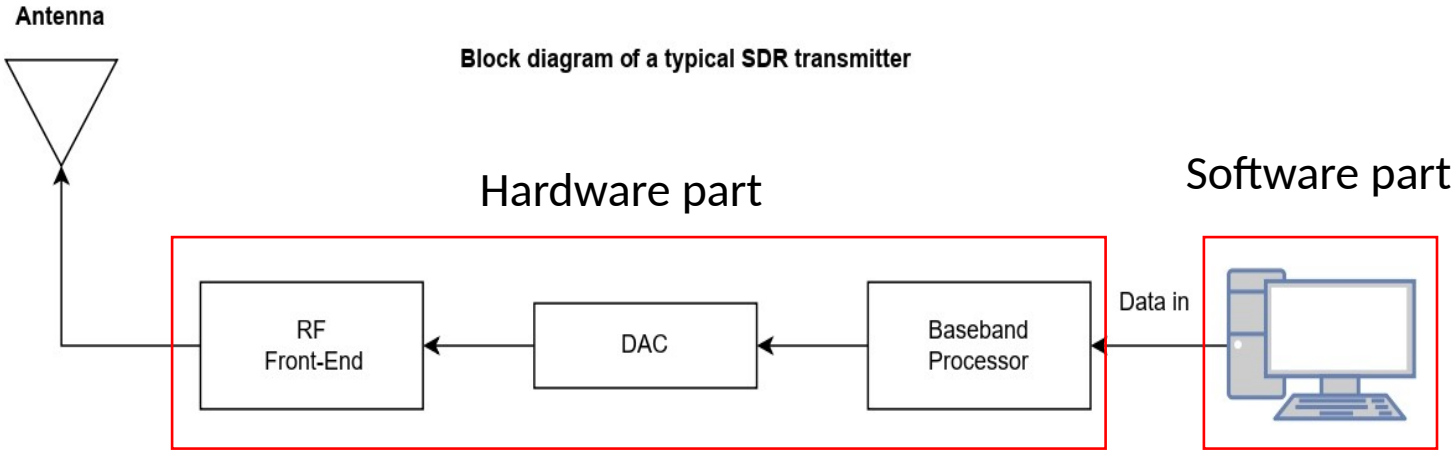
- You need a software to link the SDR device to your computer
- First you need the driver of the SDR device
- Then you have few options to demodulate and decode signals
  - Write you own software
  - Install a software sold by a company, such as Mathlab/simulink
  - Install an open software, such as GNURadio, GQRX, GNU Octave
- Our choice is to use GNURadio

Cognitive radio

- The idea is to modify the signal and its modulation according to the context
- Wi-Fi signals can be modifyied with the distance between the sender and the receiver
- Wi-Fi receiver can find the best channel with the minimum of interferences
- In case of electronic attack which corresponds to generate interferences to block communication, we can imagine that 2 cognitive radios can change their frequency, modulation in case of communication losses.

Software Defined Radio





- SDR is the process of getting code as close as possible to the antenna
- Most of functionalities are implemented in software rather than hardware
- Digitizing is the main feature of SDR system
- Hardware problems are turned into software problems

Most popular SDR devices

RTL receiver only

Hack-RF transmit and receive device, from 1 to 6000 MHz
around 350 euros

ADALM Pluto, from 325 up to 6000 MHz
around 350 €

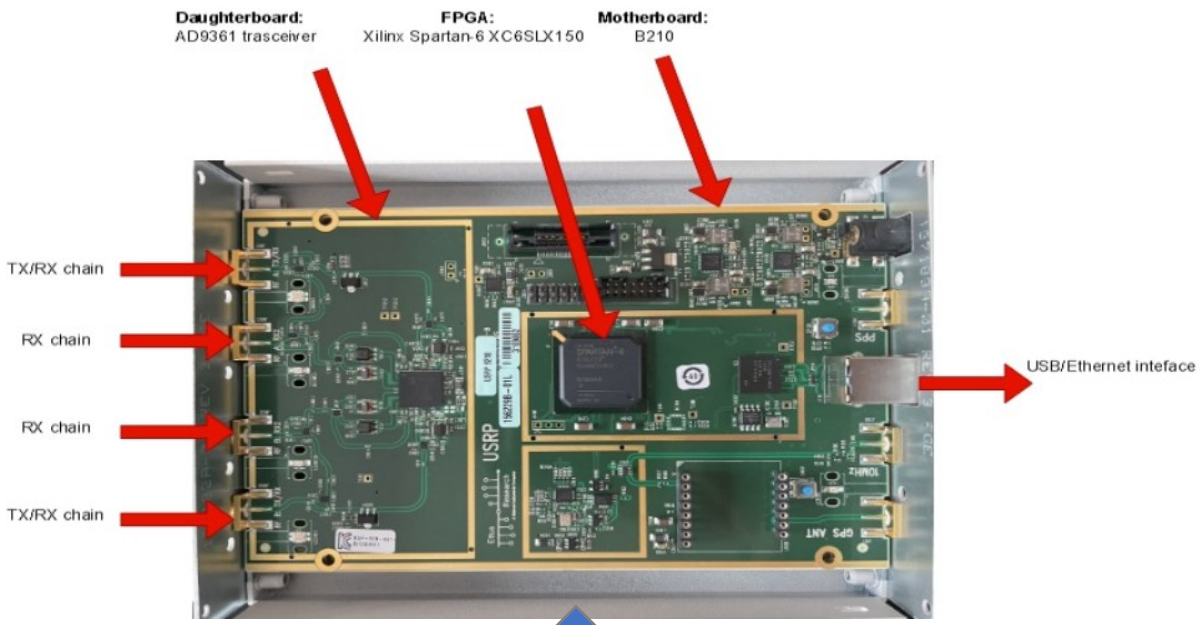
USRP B200 mini, from 70 up to 6000MHz,
around 1000 €

Most popular SDR devices

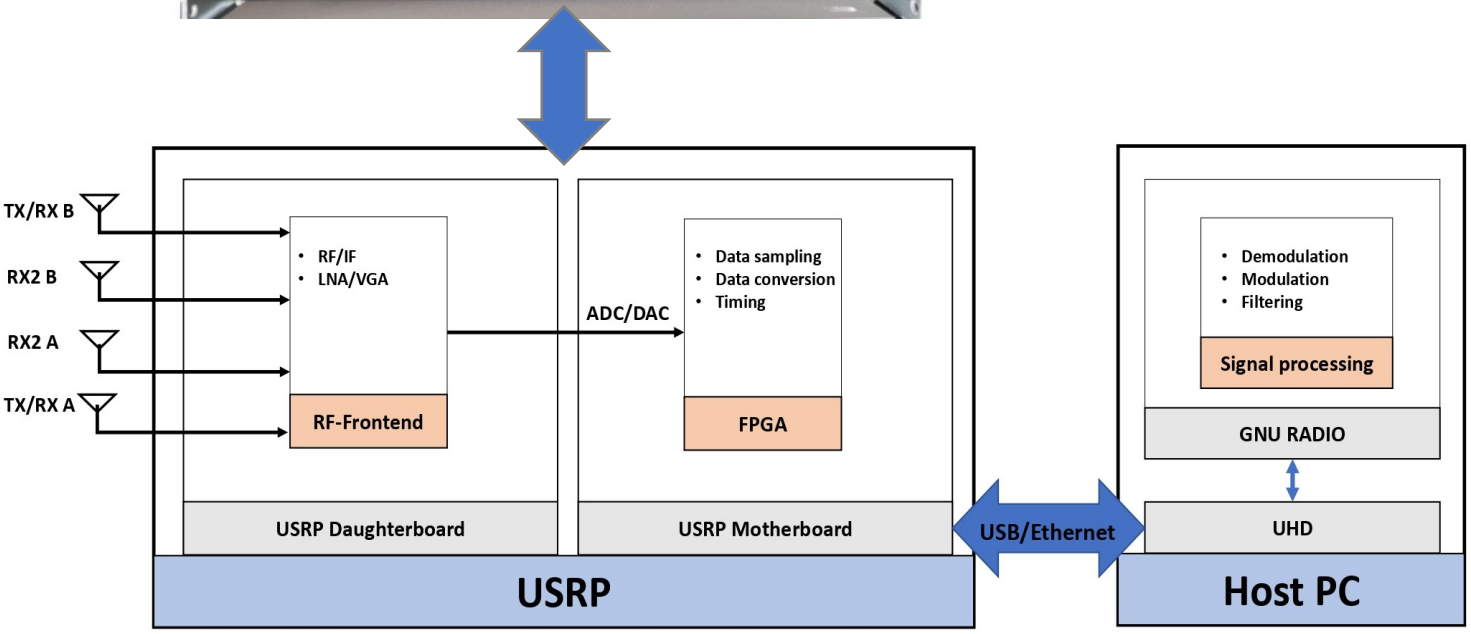
Here's a brief comparision between a few popular SDR peripheral devices:

Name	Frequency Range	Bandwidth	ADC Resolution	Tx capability	Price
RTL-SDR R820T2/RTL2838U	0.5 – 1766 MHz	Matches sampling rate, but with filter roll-off	8	NO	USD 24
HackRF One	1 MHz – 6 GHz	20 MHz	8	YES	USD 299
bladeRF	47 MHz – 6 GHz	56 MHz	12	YES	USD 480
USRP B210	70 MHz – 6 GHz	56 MHz	12	YES	USD 1,100
LimeSDR	100 kHz – 3.8 GHz	61.44 MHz	12	YES	USD 299
YARD Stick One	300 MHz – 1GHz	-	8	YES	USD 100

Software Defined Radio/USRP

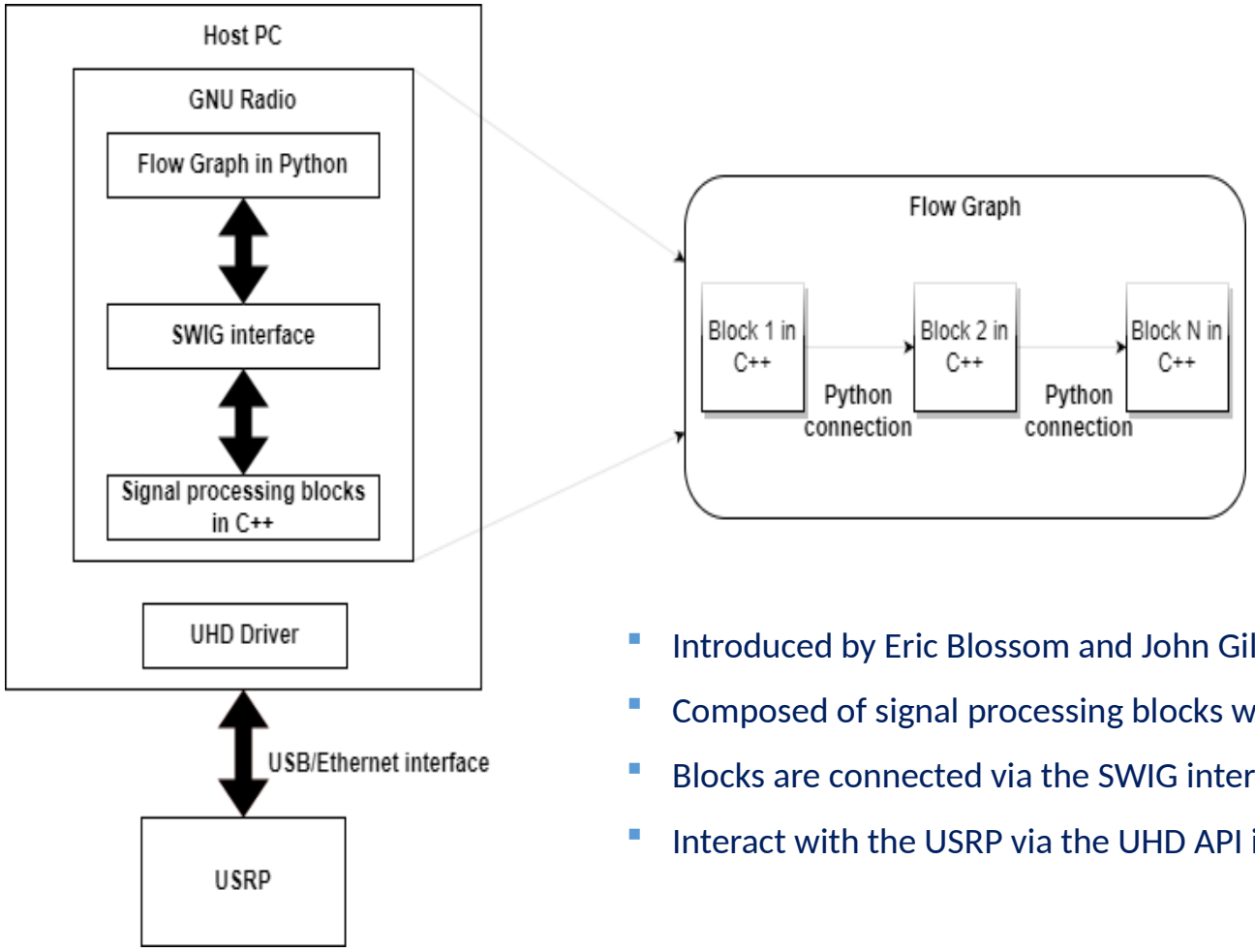


- USRP is the most popular cost-effective, high-quality components, and accurate SDR platform used in research applications
- Reasonable prices compared to other systems
- High flexibility, accuracy, phase and frequency stability
- Large bandwidth that can capture many kinds of signals



- Wide frequency range (70 MHz to 6 GHz)
- Maximum bandwidth of 56 MHz
- Sampling rate of 61.44 Msps
- ADC of 12 resolutions bits
- AD9361 daughterboard
- Xlinx Spartan 6 FPGA
- USB2.0/3/0 and Ethernet

Software Defined Radio/GNU Radio



- Introduced by Eric Blossom and John Gillmore in 2001 as a part of GNU package
- Composed of signal processing blocks written in C++
- Blocks are connected via the SWIG interface written in Python to use C/C++ functions
- Interact with the USRP via the UHD API interface

GNURadio

Introduction:

GNU Radio is an open-source **toolbox** for building software radios. Unlike conventional radios, it's not a fix hardware that defines the operations applied to the signals, but a highly modular software.
GNU Radio is a **signal processing** environment including hundreds of signal processing and digital communications blocks, all developed in C++ or Python. It allows you to **transmit** and **receive** sampled signals using cards such as those sold by Ettus or NI (**USRP card**), or to receive demodulated signals using **USB sticks DVB-T tuner** using the component RTL2832U+R820T

GNU Radio

GNU Radio provides libraries of Signal Processing Modules (SPMs) for tasks specific to software radios: **modulations** (PSK, QAM, etc.), OFDM-type **spread spectrum functions, error-correcting codes** (Reed-Solomon, Viterbi, Turbo Codes), signal processing functions (filters, FFTs, equalizers), clock recovery, input/output operations such as file access, etc.

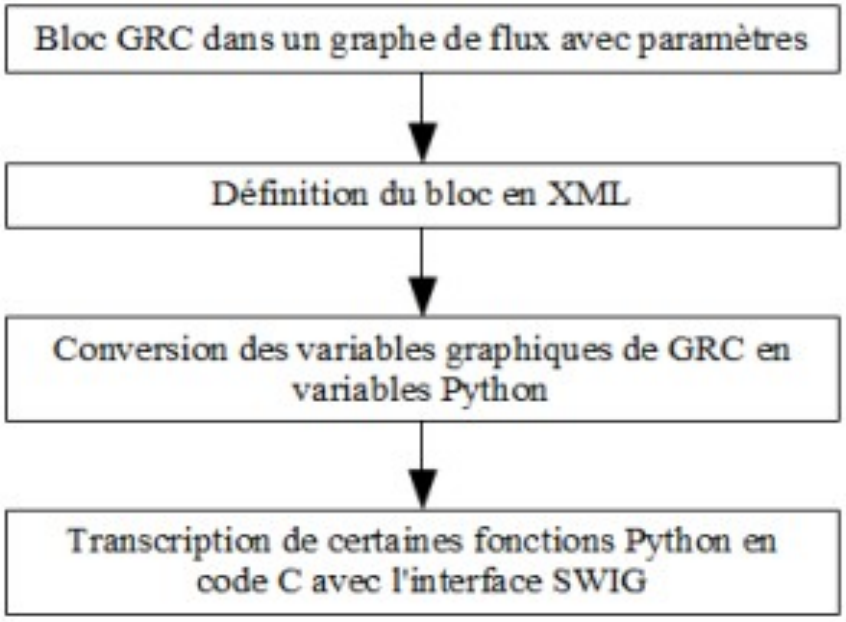
GNURadio

GRC (GNURadio companion):

GRC (GNU Radio Companion) is an application distributed with GNU Radio that can be used to create software radios with flow graphs using a number of predefined function blocks (source signals, outputs or sinks, modulation or demodulation functions, etc.). GRC is a 'drag and drop' graphical interface dedicated to the design of GNU Radio transmission chains.

GNURadio

The signal processing blocks are graphically represented with their parameters. GRC automatically generates the equivalent Python code from the diagram produced.



GRC block in a flow graph with parameters

Description of the block in XML

Graphical variables from GRC are converted into Python variables

Some Python functions are translated into C code using SWIG interface

GNURadio

GNU Radio QT GUI Signal Analysis Tools

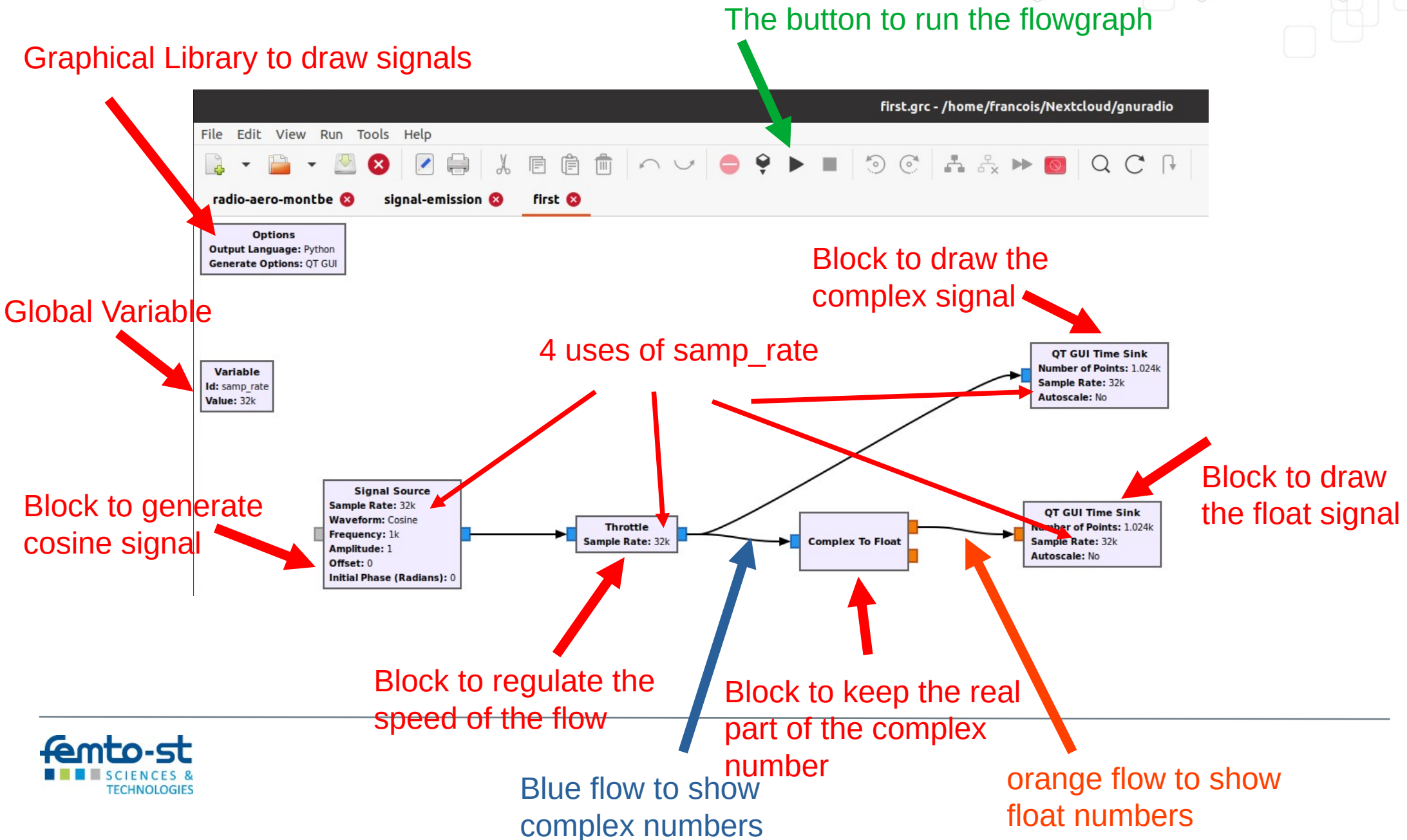
GNU Radio QT GUI Signal Analysis Tools enhances GNU Radio's graphical interface by adding signal analysis functionalities. It features a spectrum analyzer, 2D spectrograph (waterfall), oscilloscope, histogram and constellation diagram...

GNURadio

Programming Languages:

Using Python, you can easily write a windowed application using WT/QT graphics, or interact with modules written in C/C++. Since Python is interpreted, it is independent of the operating system used.
GNU Radio development involves building a communication system by creating a **signal processing graph** (or flow graph) where the nodes are the **signal processing blocks** and the branches represent the **data flow** between the blocks. The low-level blocks are implemented in C++ , while the high-level blocks and GNU Radio graphs are created in Python .
The whole system is orchestrated by a Python scheduler.

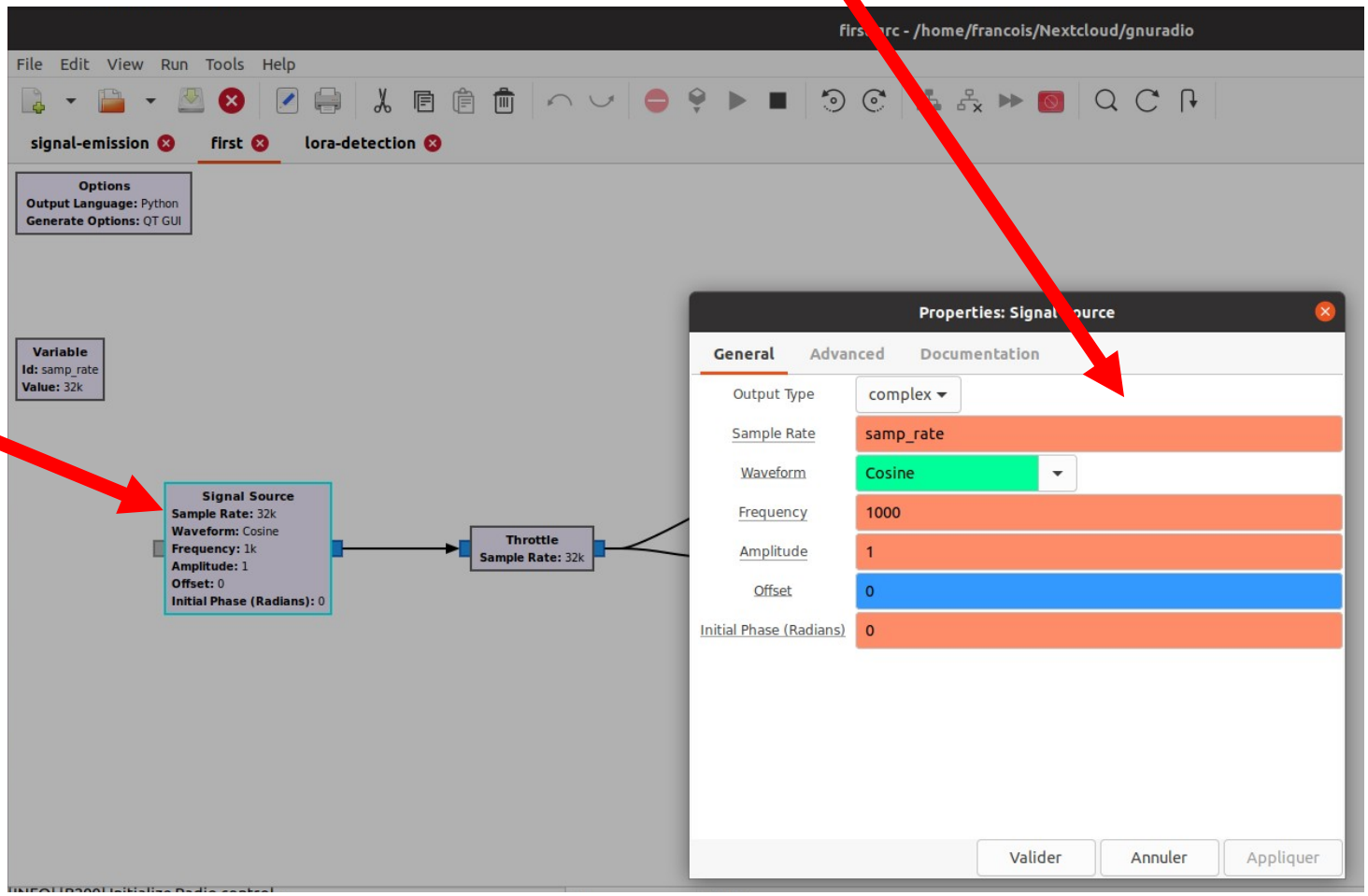
Generate a mathematic signal with GNU Radio



Generate a mathematic signal with GNU Radio

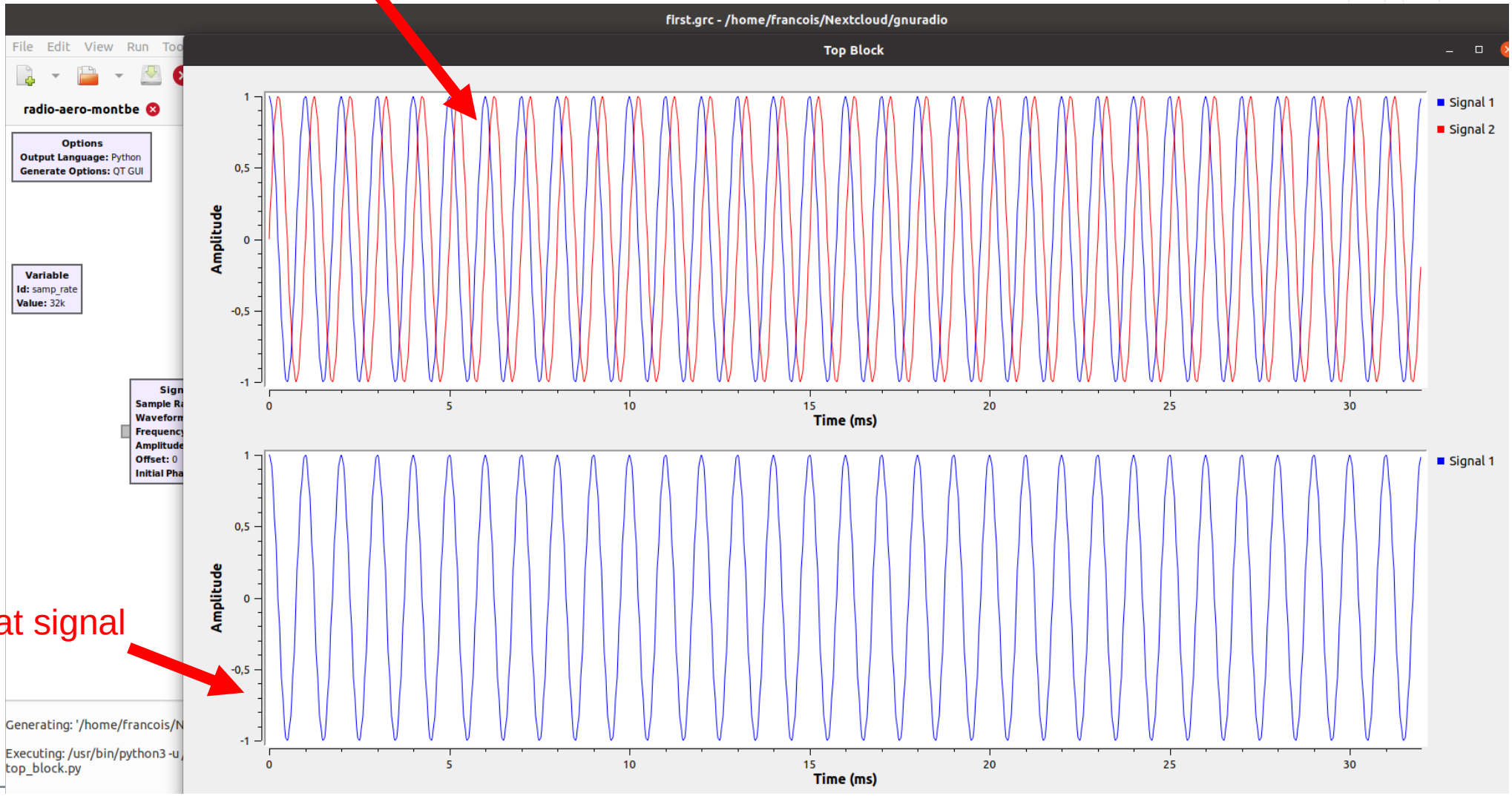
In the opened window you can change all the fields

Double click on the block allow to modify any field



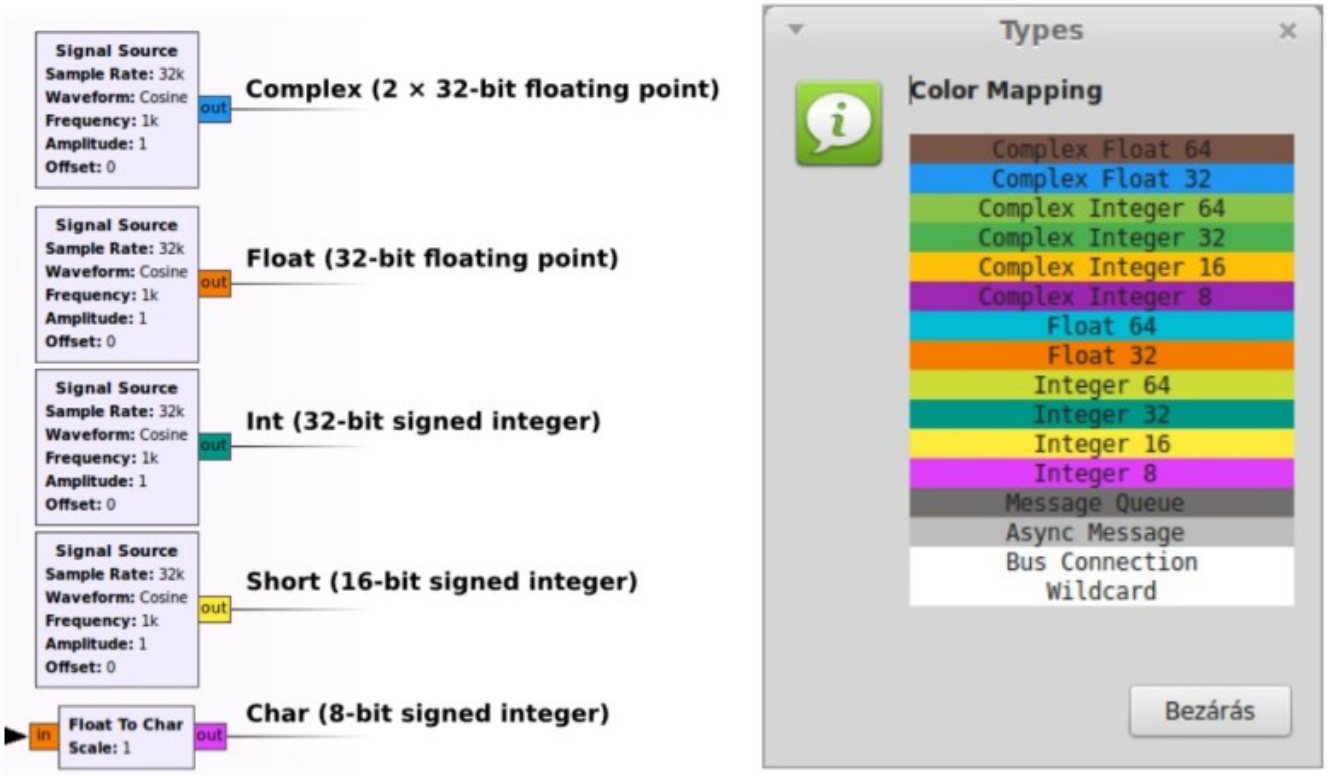
Generate a mathematic signal with GNU Radio

In Blue the I signal and in red the Q signal of the complex flow



The float signal

Color type of the flow in GNU Radio

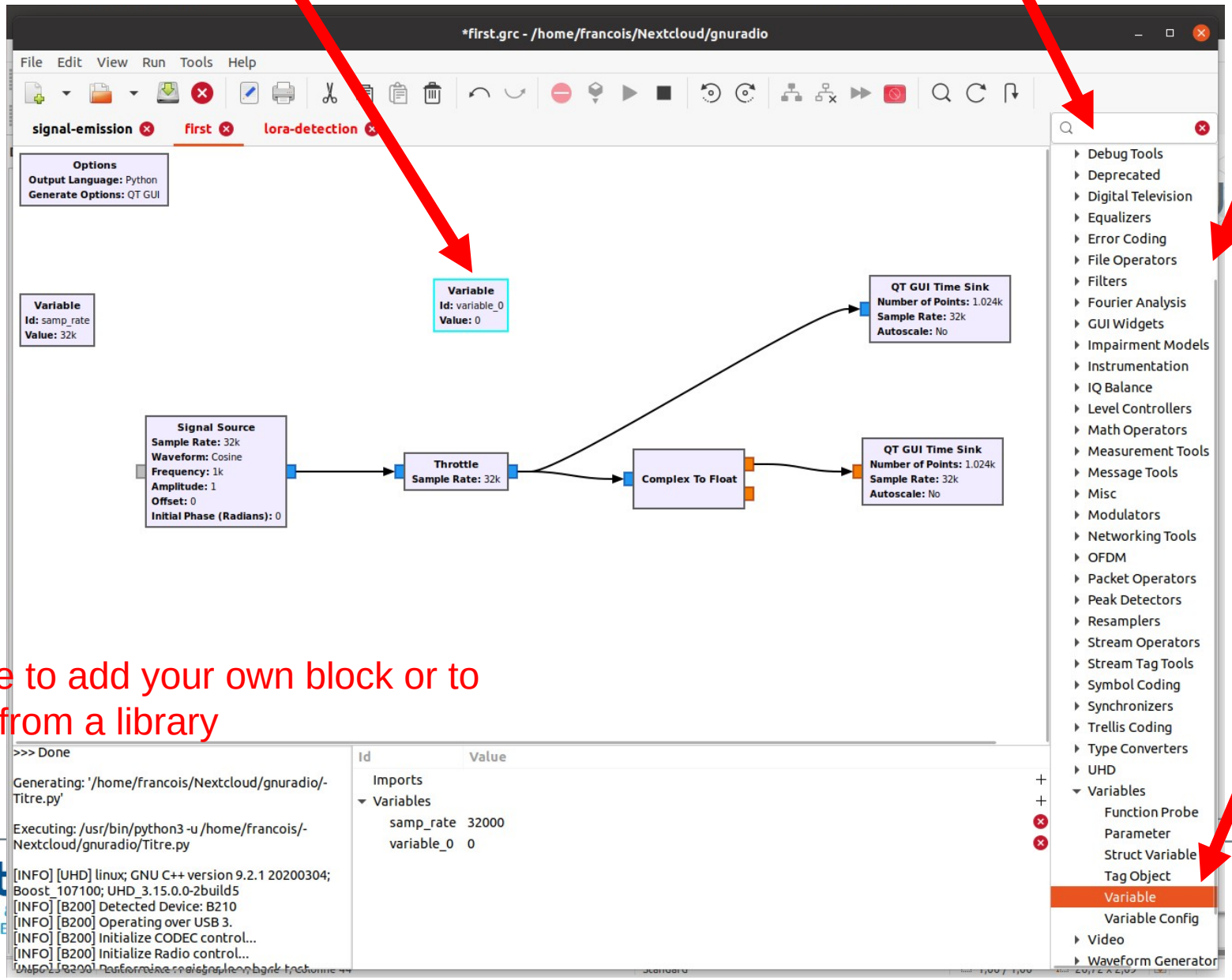


Adding a global variable block in GNU Radio

Added block for a new variable

Tool to search for a specific block

Complete list of all the blocks



It is possible to add your own block or to add blocks from a library

Block for adding a new variable

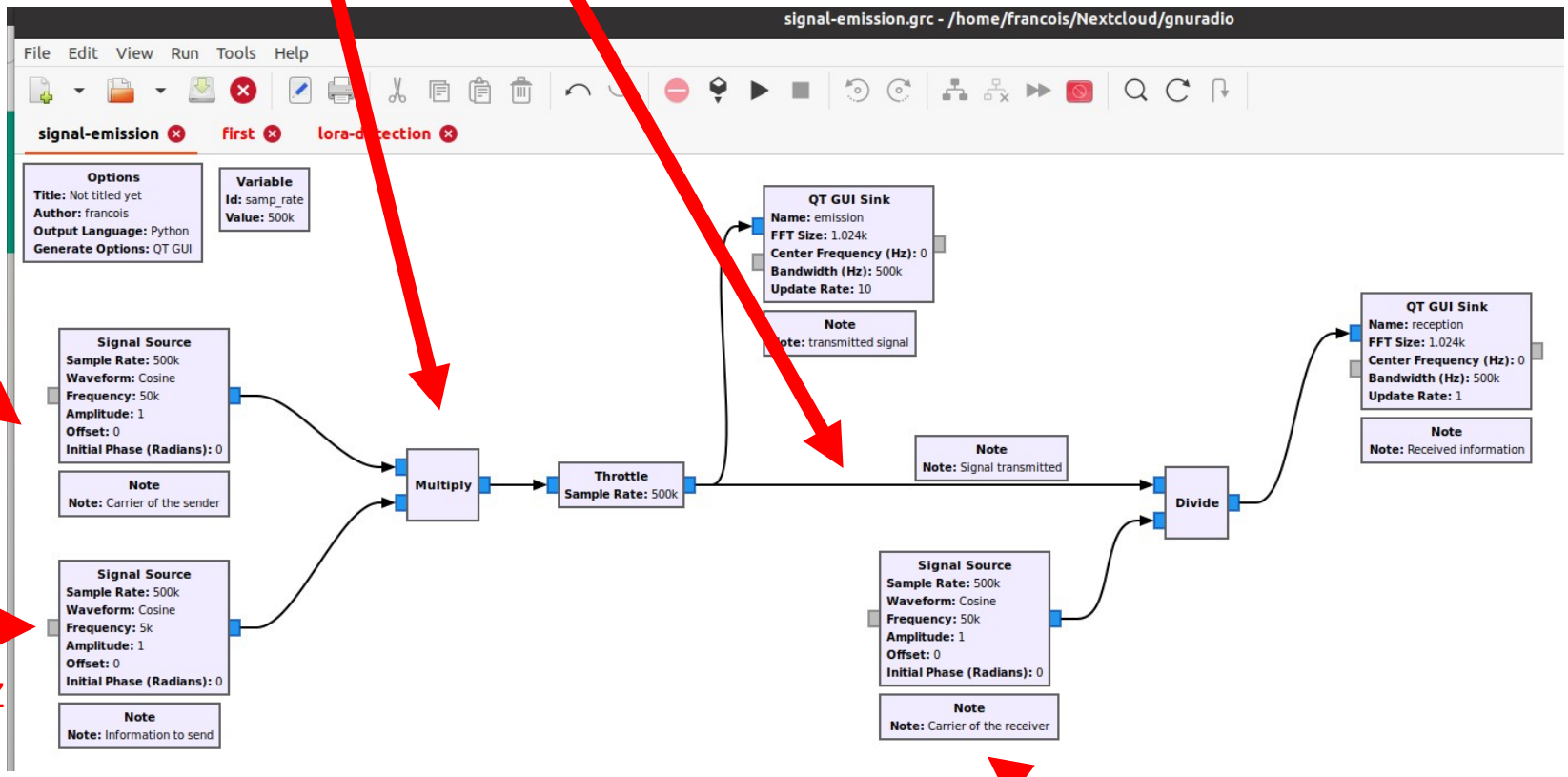
Simulate a sender and a receiver

Mathematical operator

Flow between the sender and the receiver

Carrier = 50 kHz

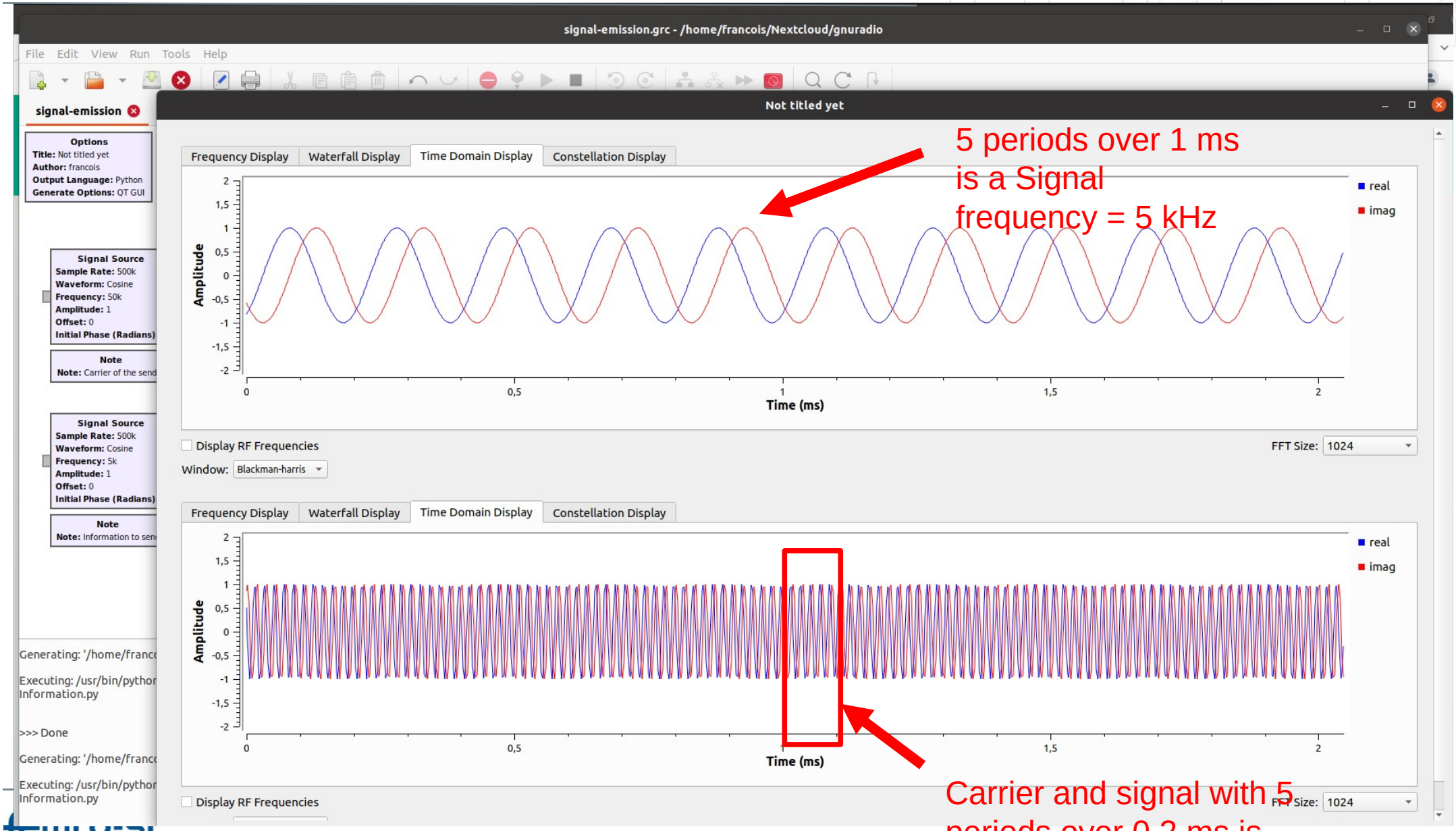
Signal frequency = 5 kHz



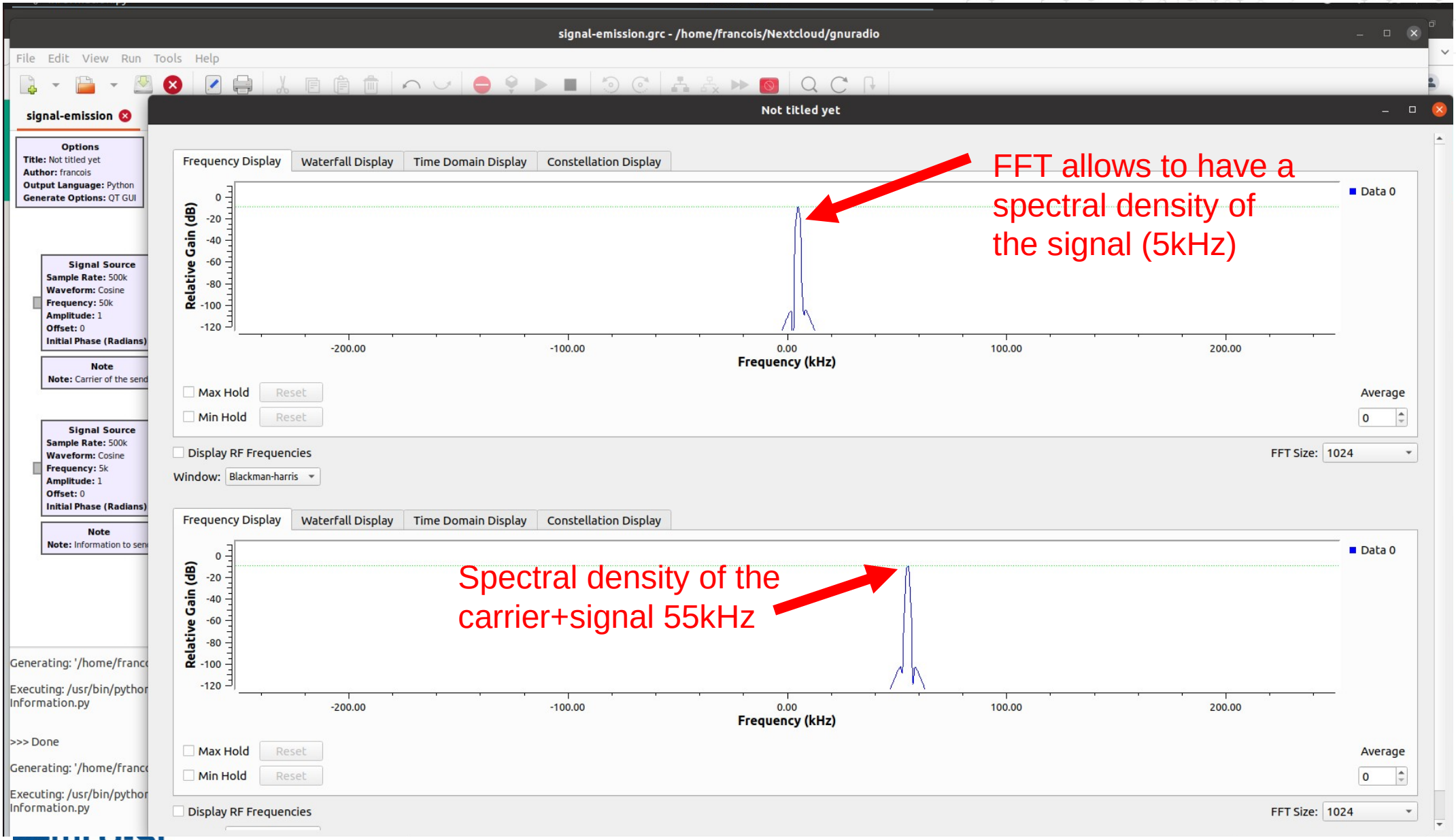
Carrier of the receiver to extract the signal

femto-st SCIENCES & TECHNOLOGIES

Simulate a sender and a receiver

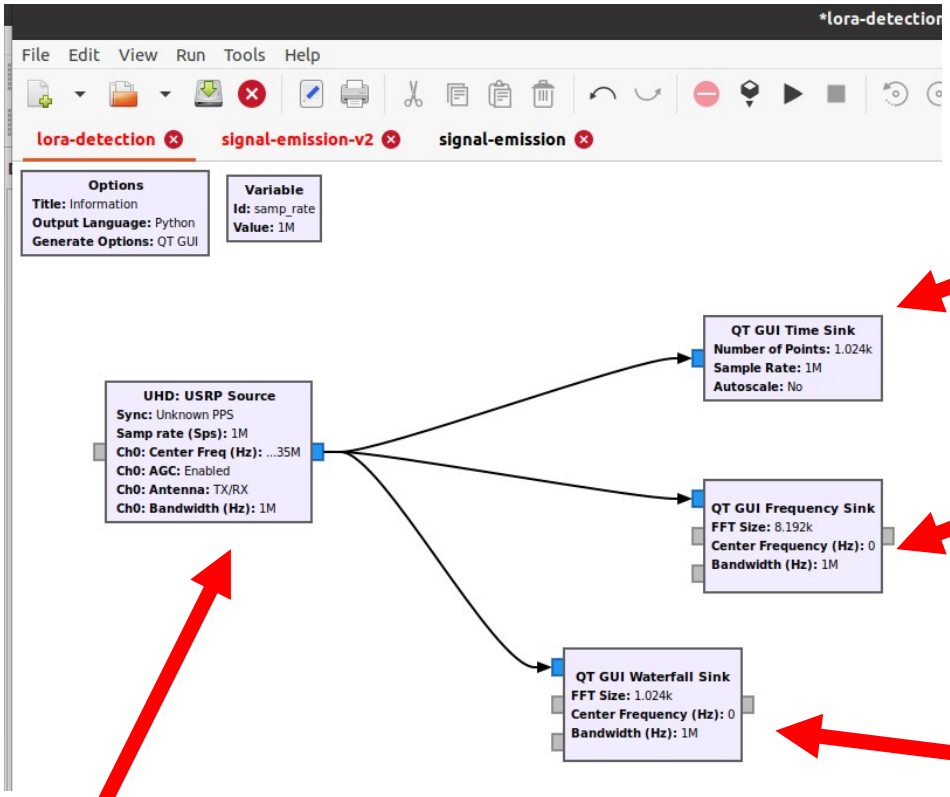


Simulate a sender and a receiver



SCIENCES & TECHNOLOGIES

Receiving a LoRa signal with GNU Radio



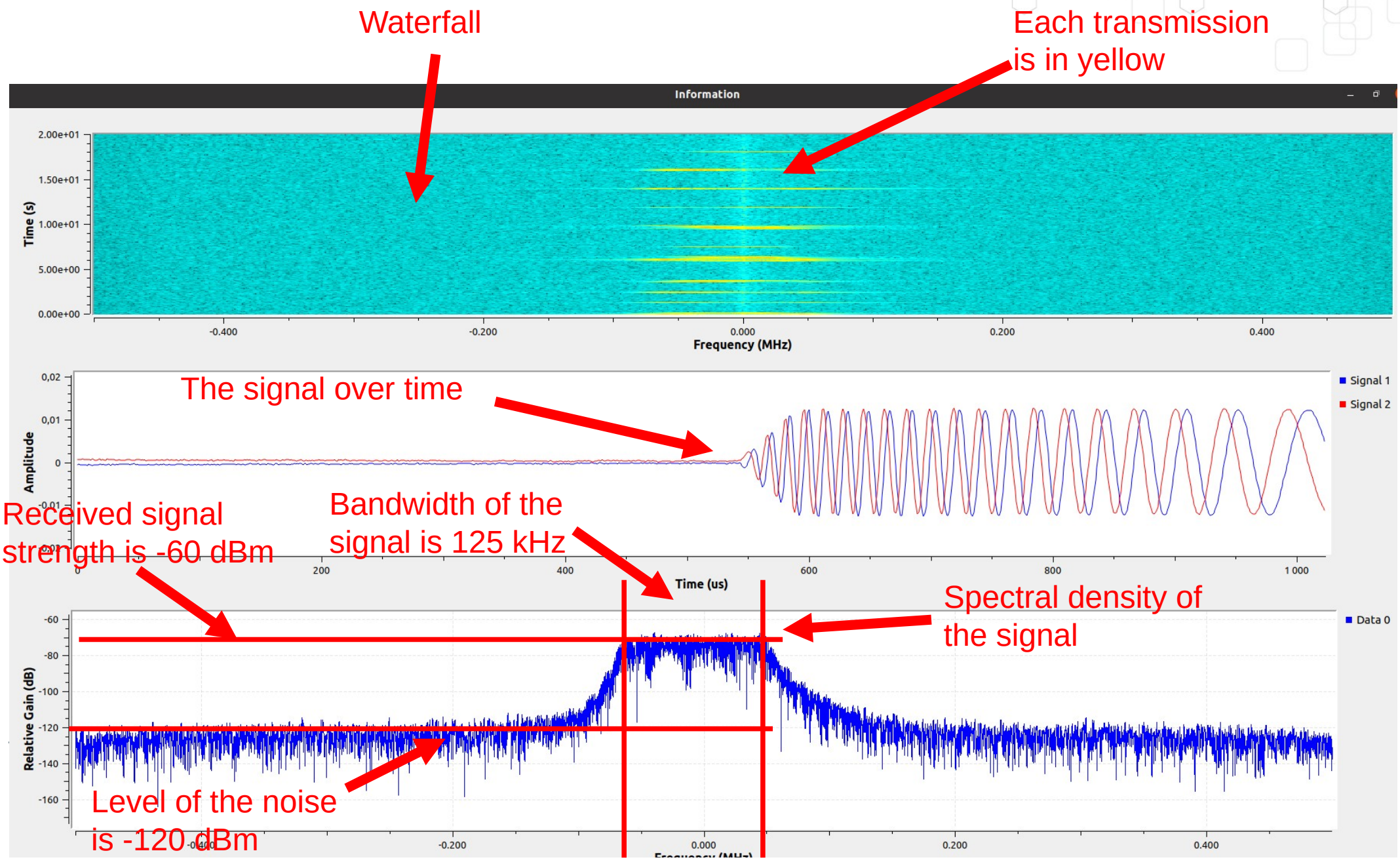
Time presentation of the signal

FFT allows to have a spectral density of the signal

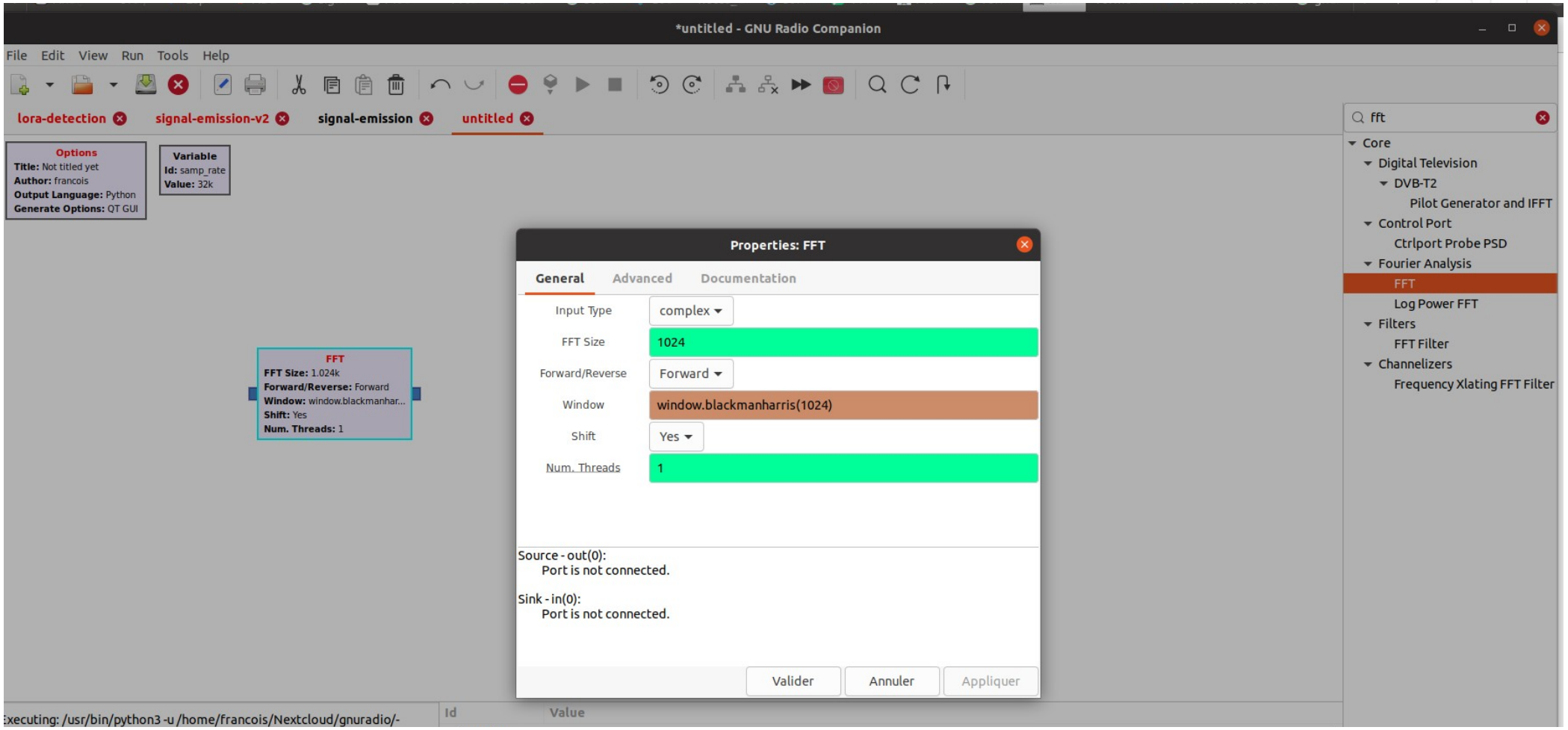
Waterfall allow to visualise transmit period and quiet period

UHD is the driver for the USRP SDR hardware.
The frequency is 868 MHz

Receiving a LoRa signal with GNU Radio

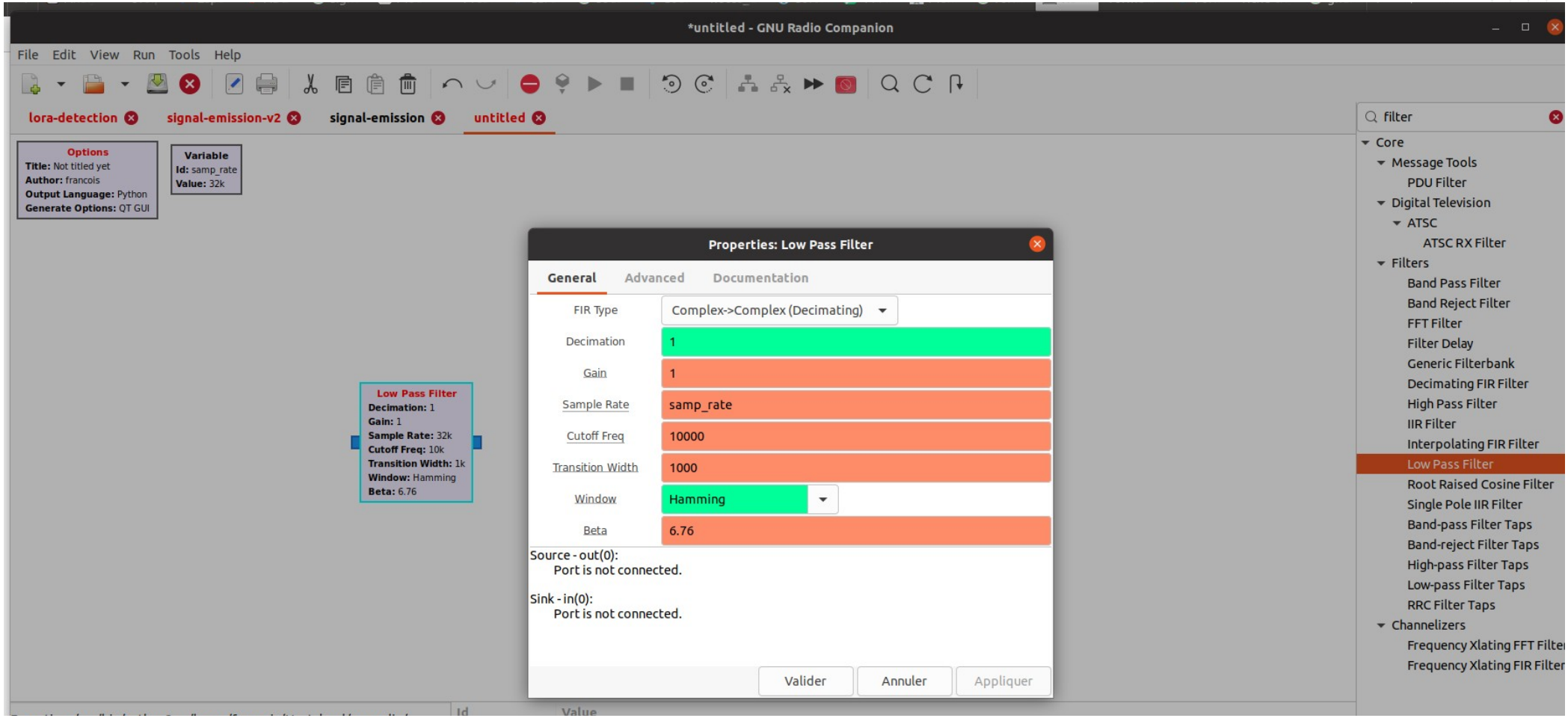


The Fast Fourier Transform Block in GNU Radio

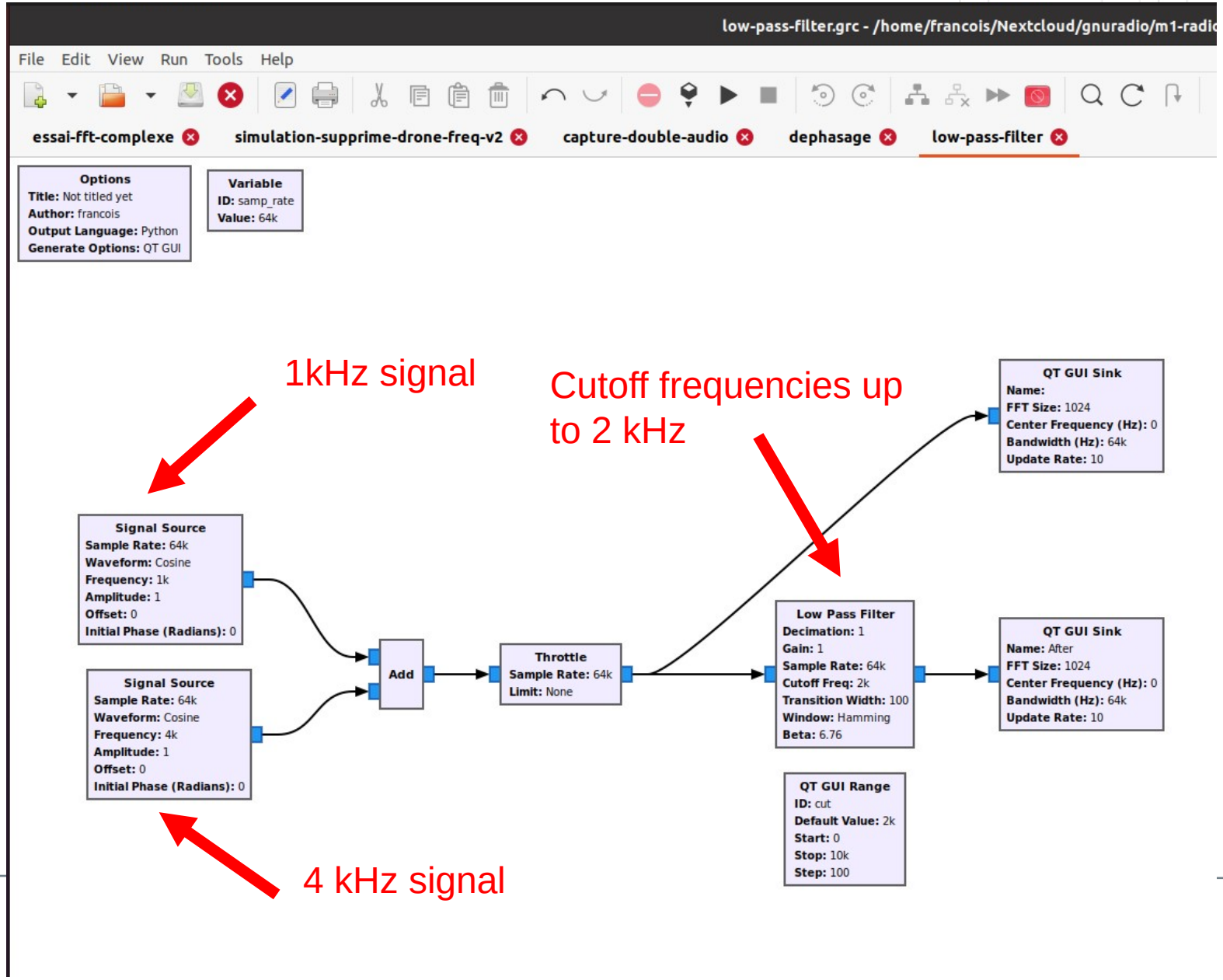


femto-st SCIENCES & TECHNOLOGIES

The Low Pass Filter Block in GNU Radio

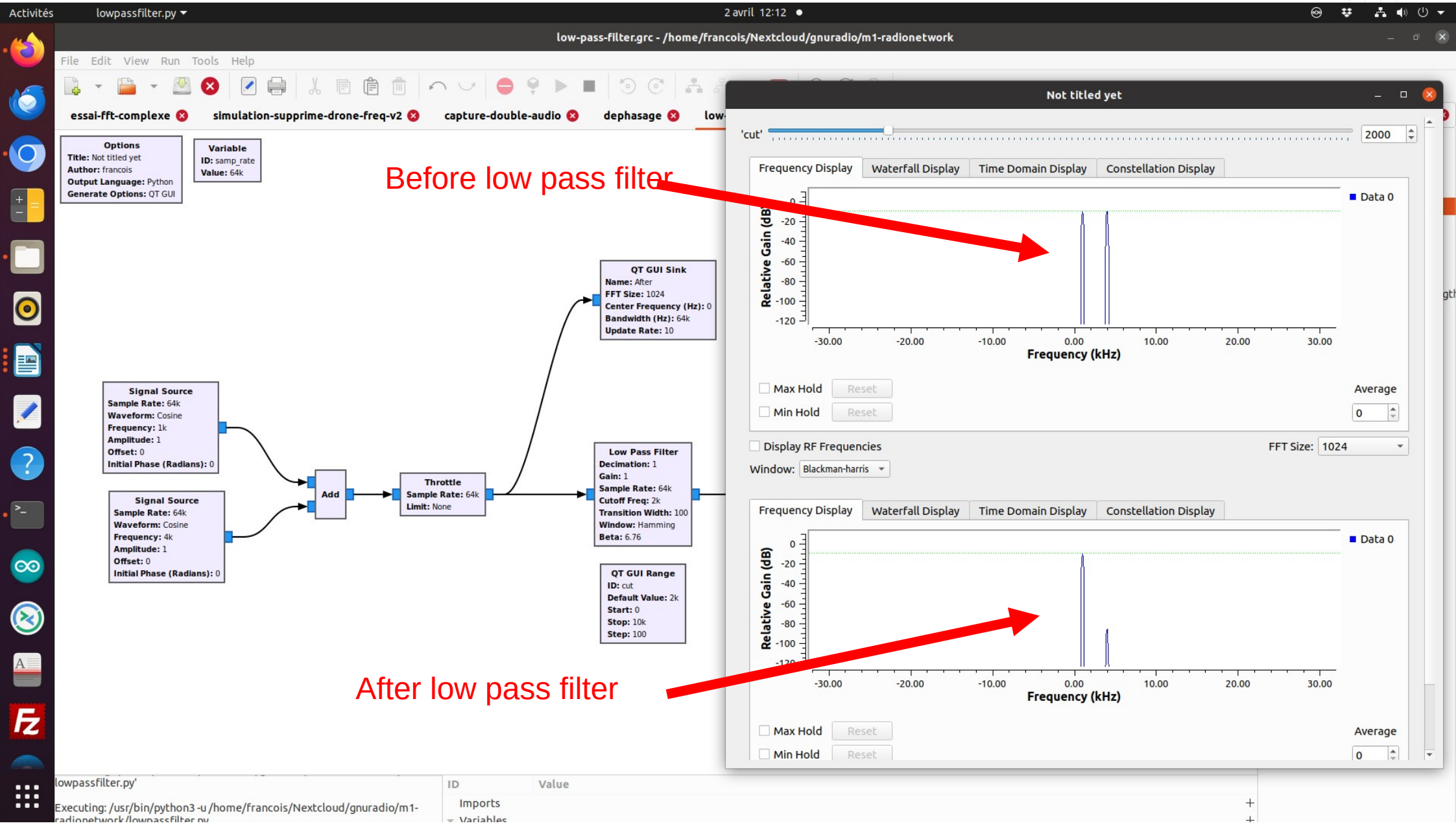


The Low Pass Filter Block in GNU Radio

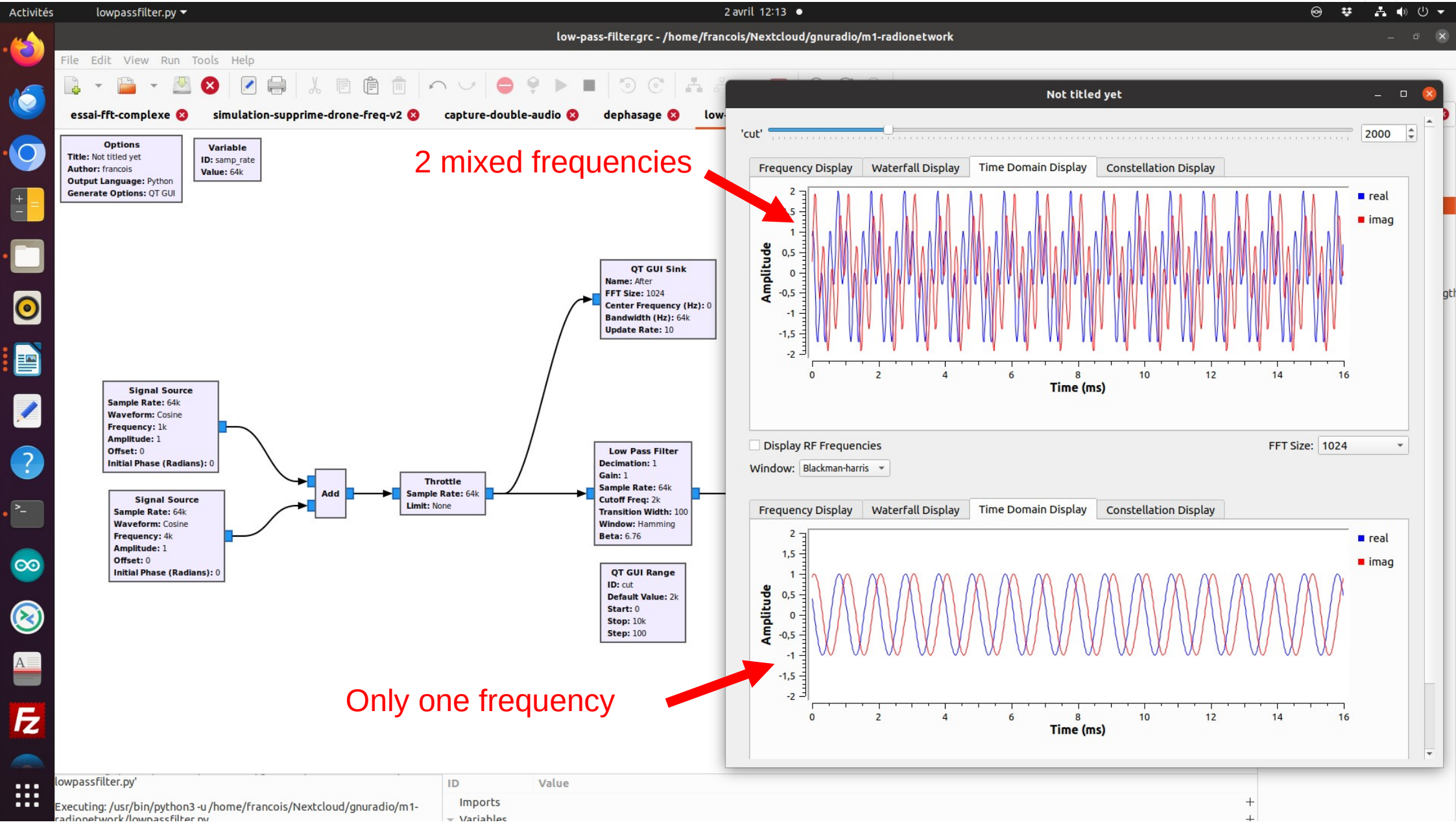


femto-st SCIENCES & TECHNOLOGIES

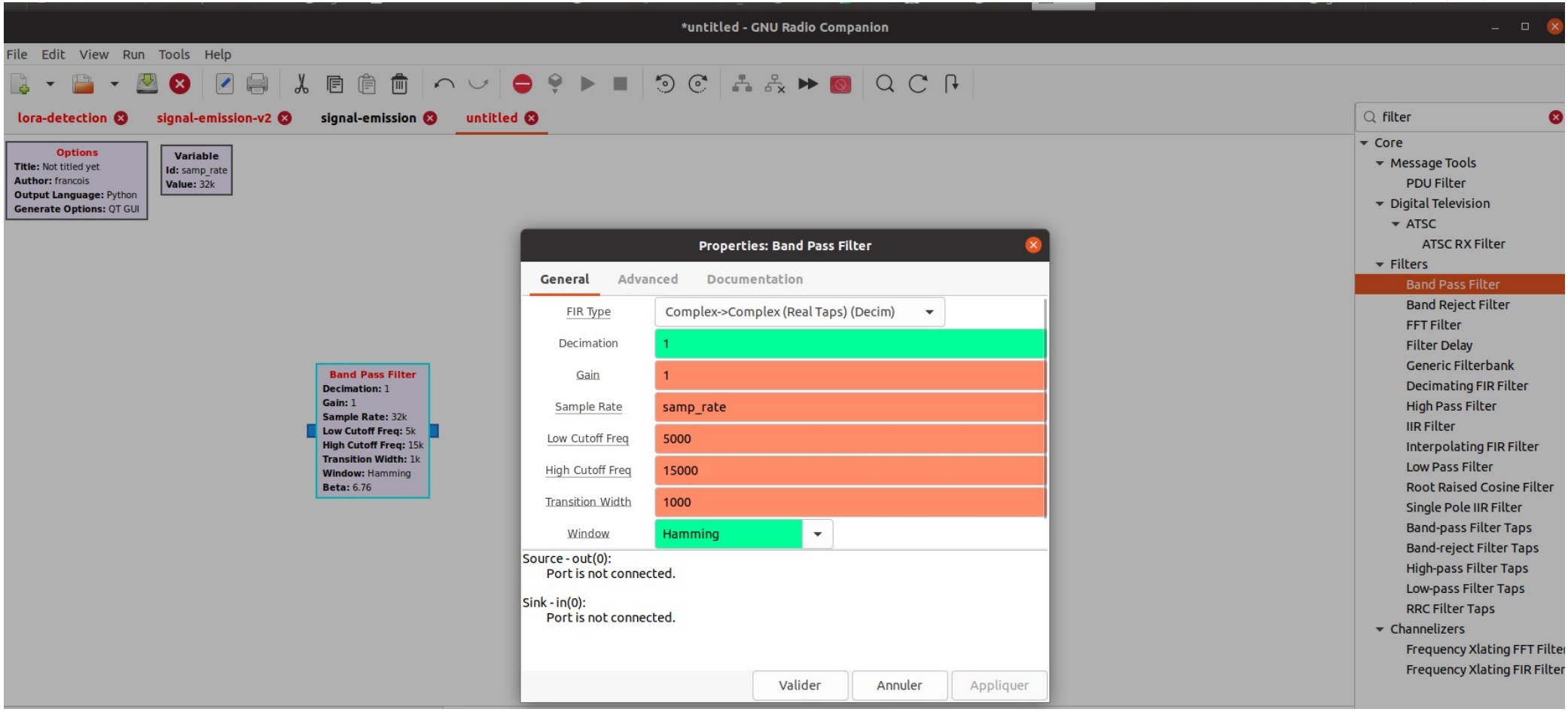
The Low Pass Filter Block in GNU Radio



The Low Pass Filter Block in GNU Radio

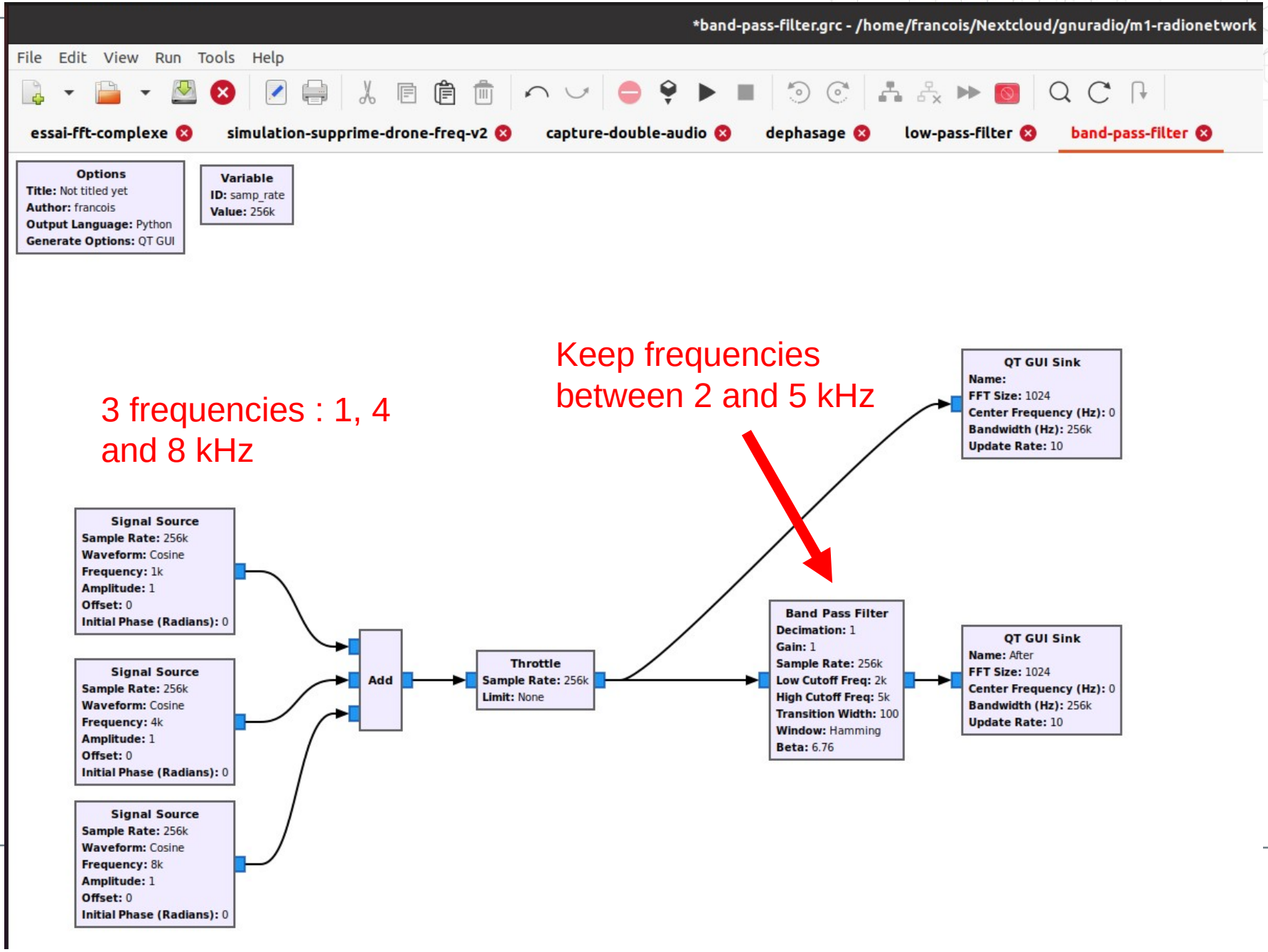


The Band Pass Filter Block in GNU Radio



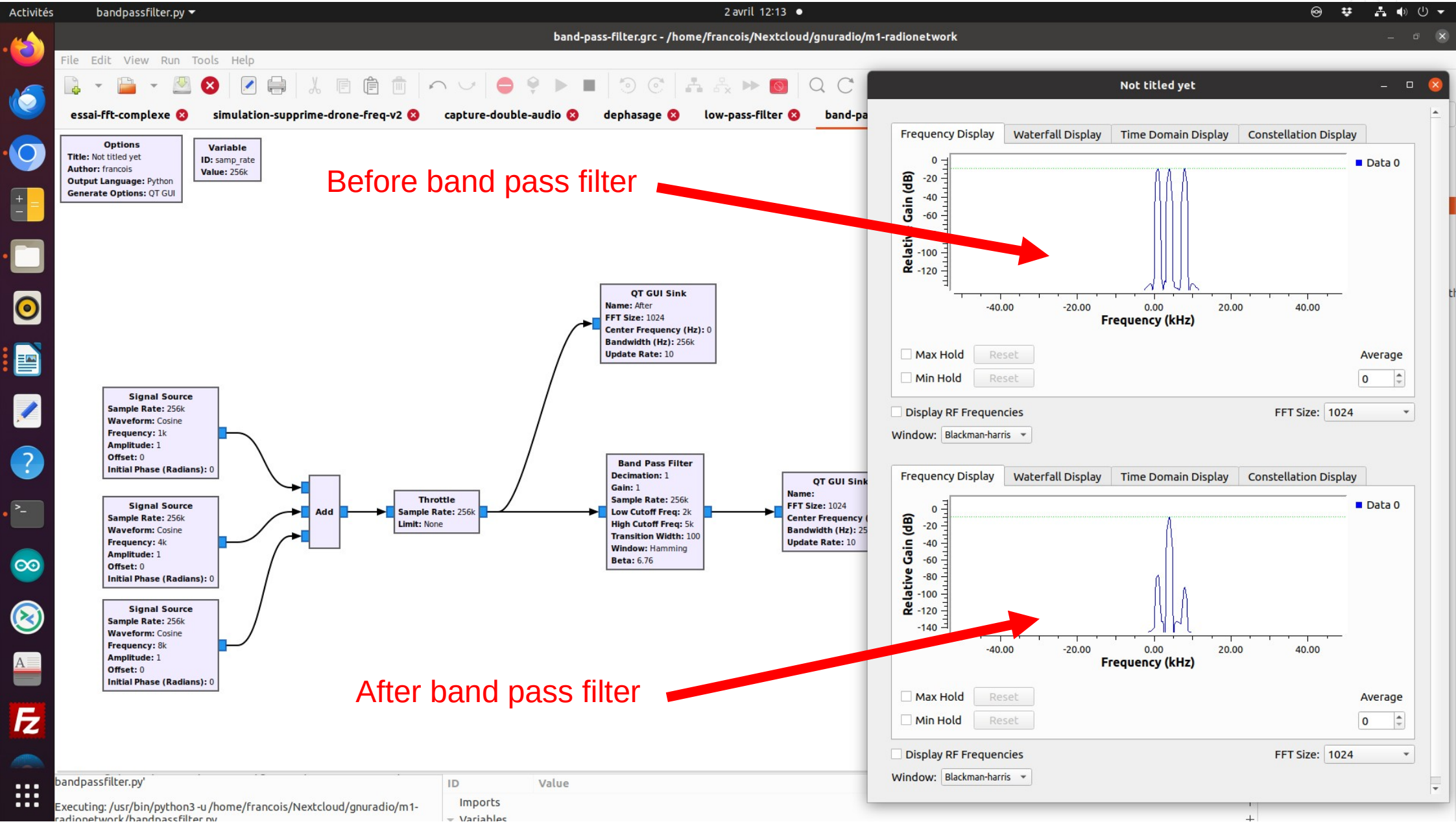
femto-st SCIENCES & TECHNOLOGIES

The Band Pass Filter Block in GNU Radio

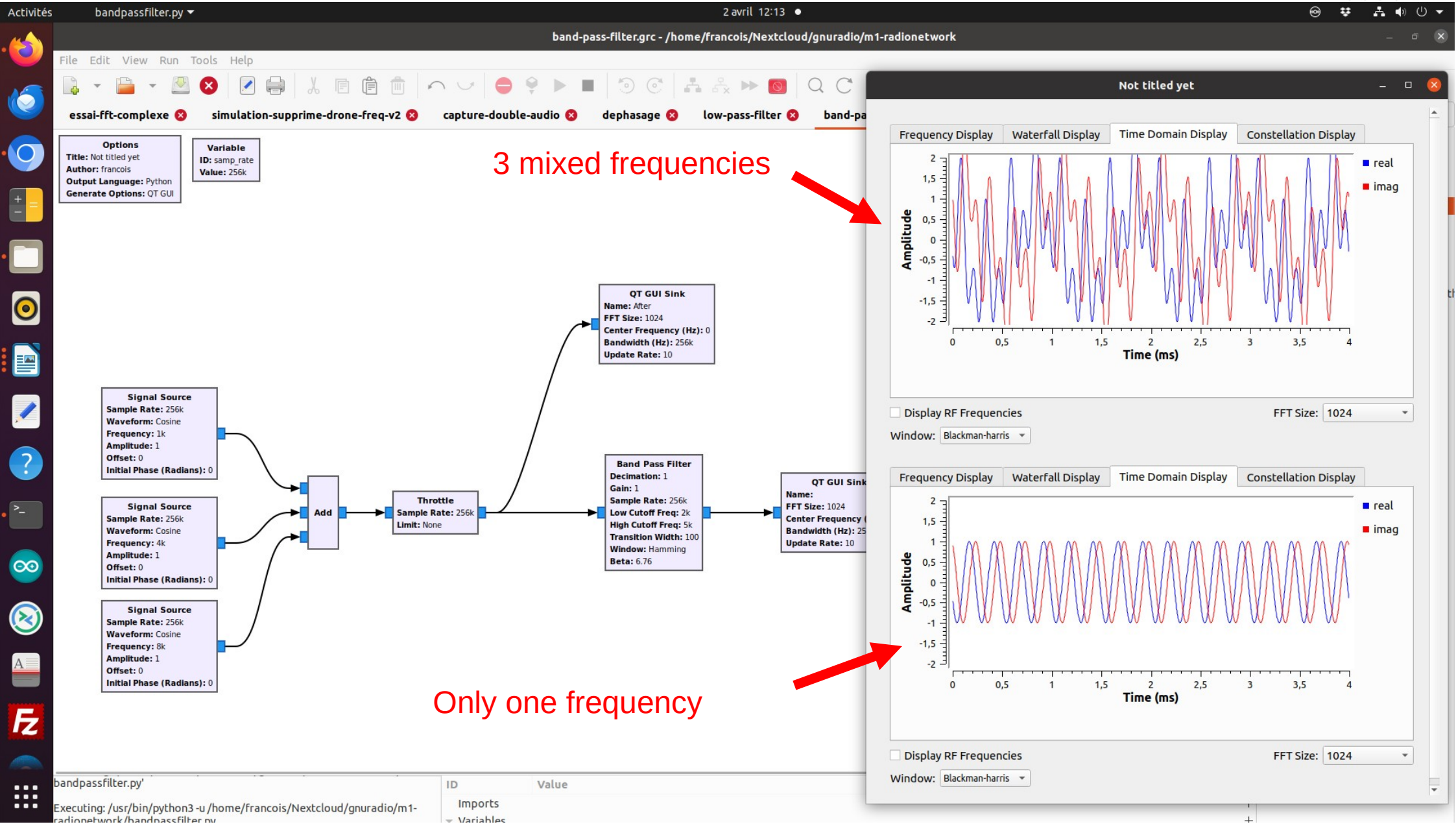


femto-st SCIENCES & TECHNOLOGIES

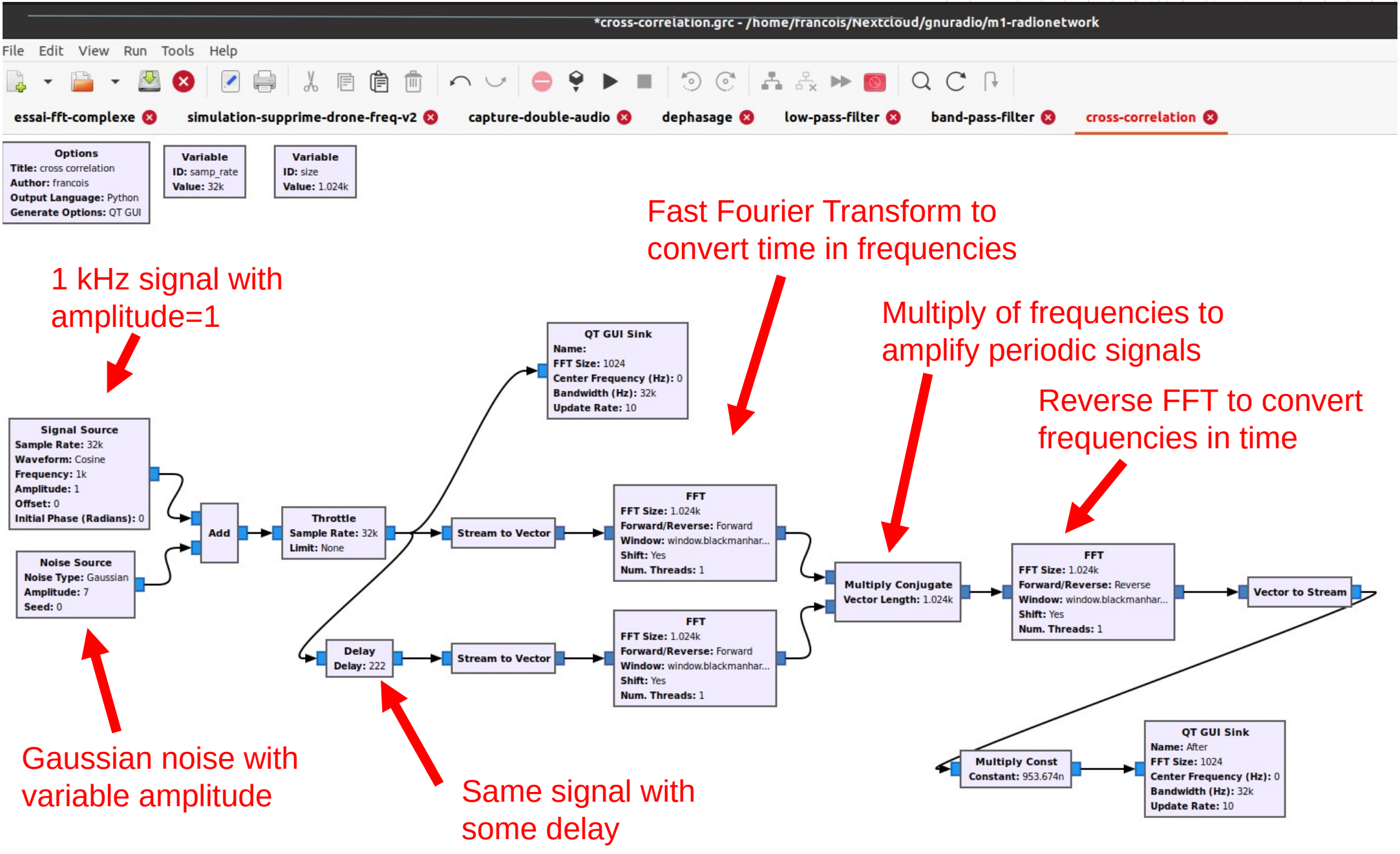
The Band Pass Filter Block in GNU Radio



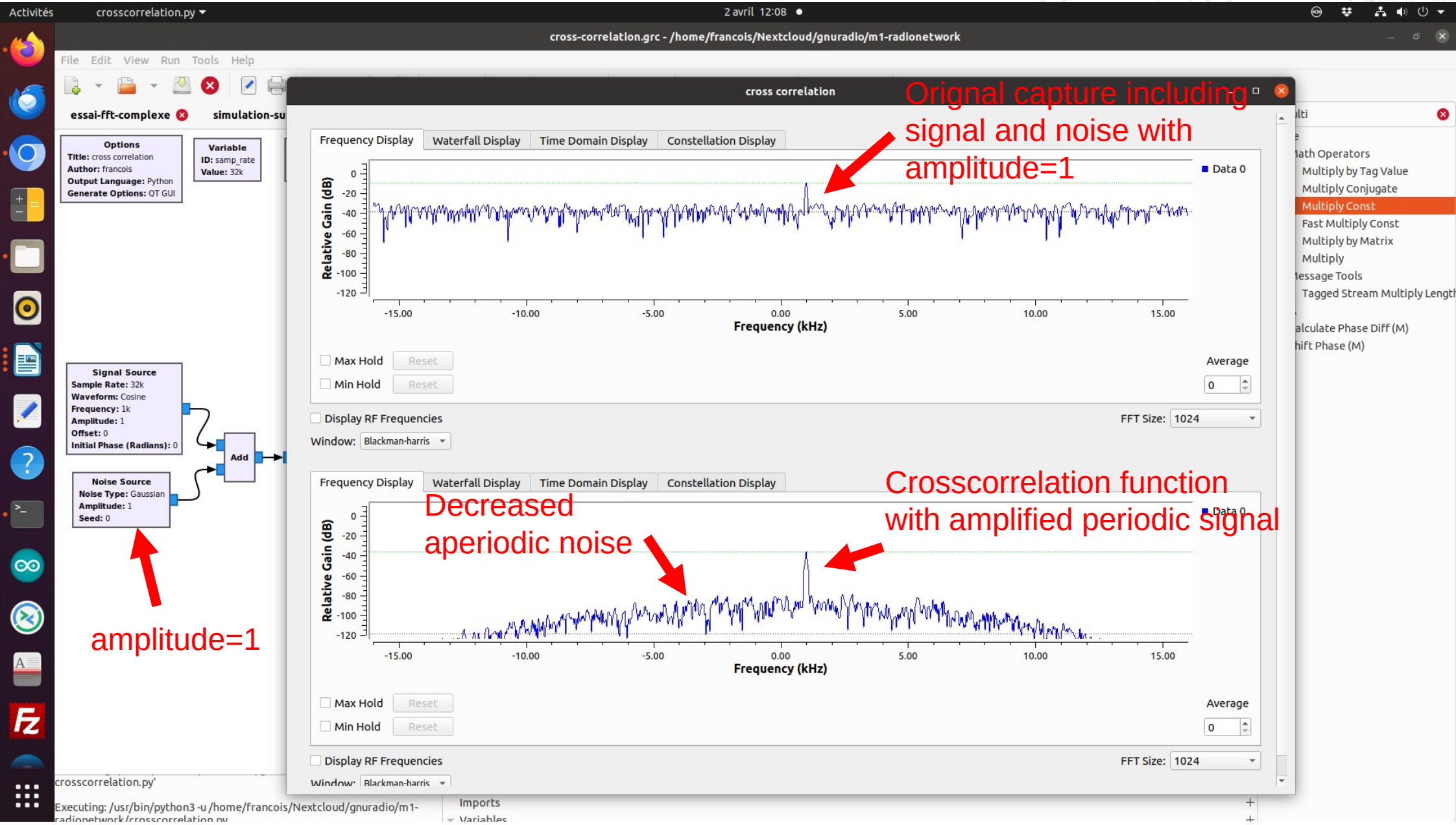
The Band Pass Filter Block in GNU Radio



Crosscorrelation in GNU Radio



Crosscorrelation in GNU Radio



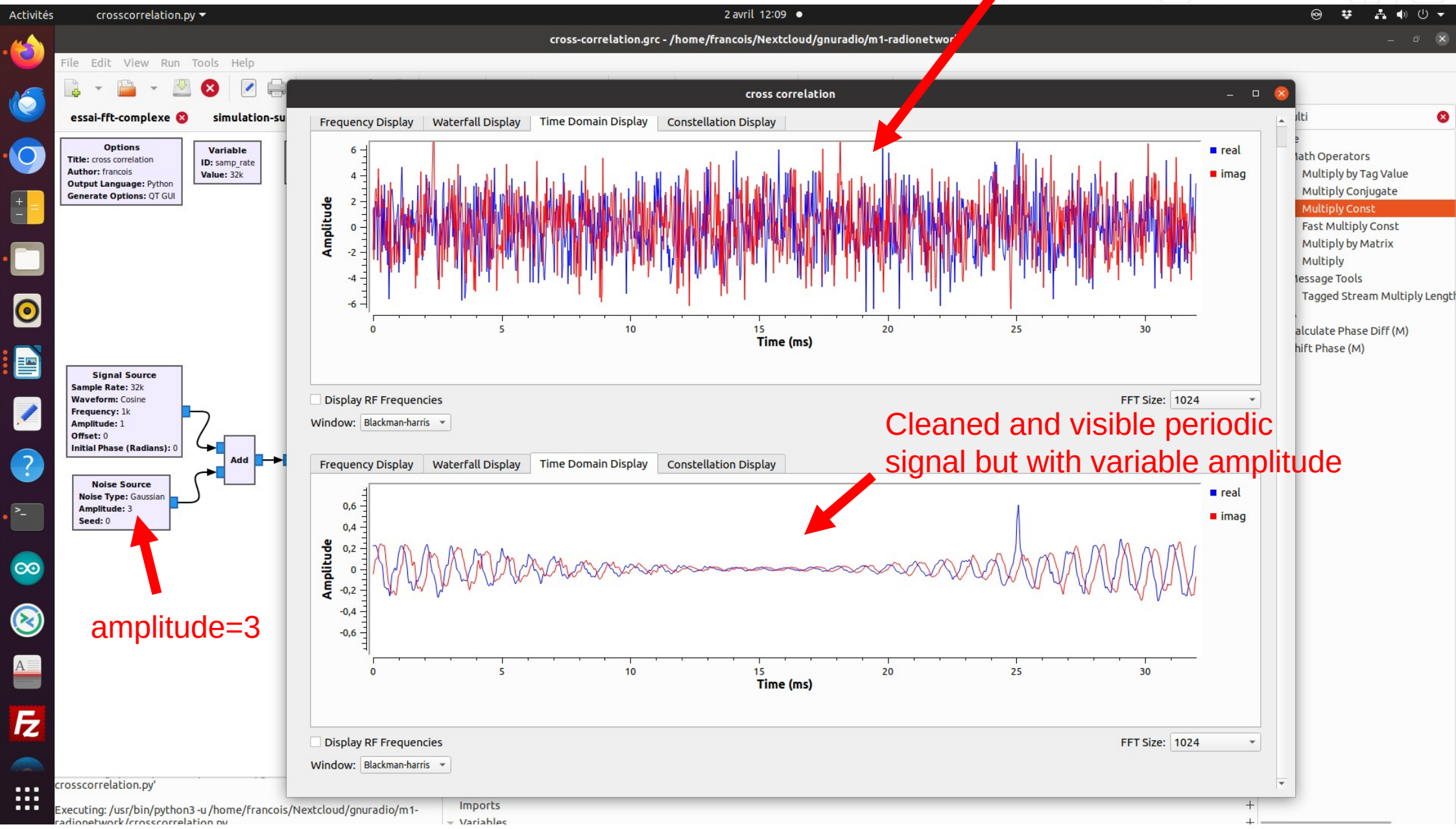
Crosscorrelation in GNU Radio

Crosscorrelation in GNU Radio

Orignal capture including signal and noise with amplitude=3

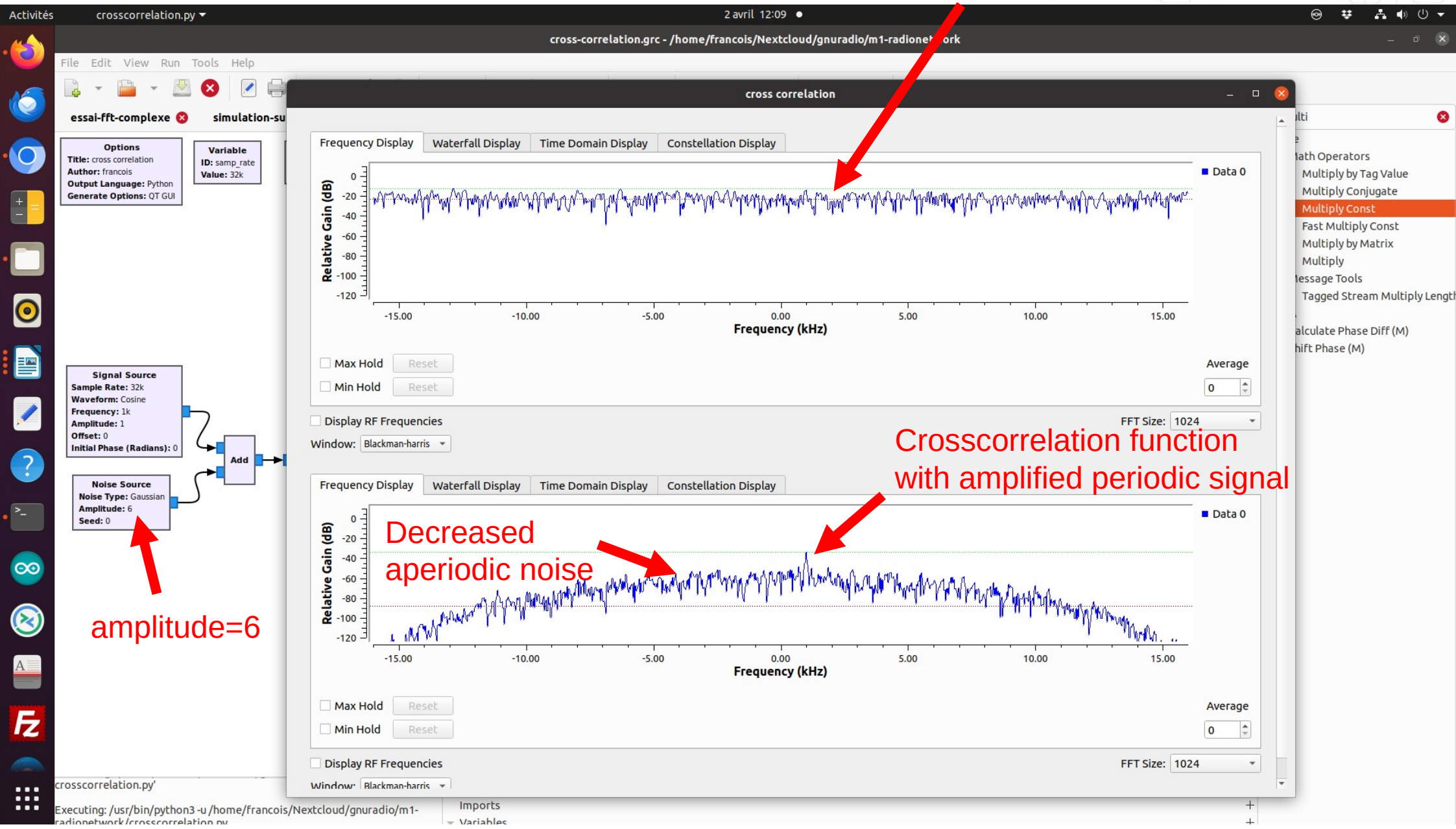
Crosscorrelation in GNU Radio

Mixed Signal and noise
without crosscorrelation

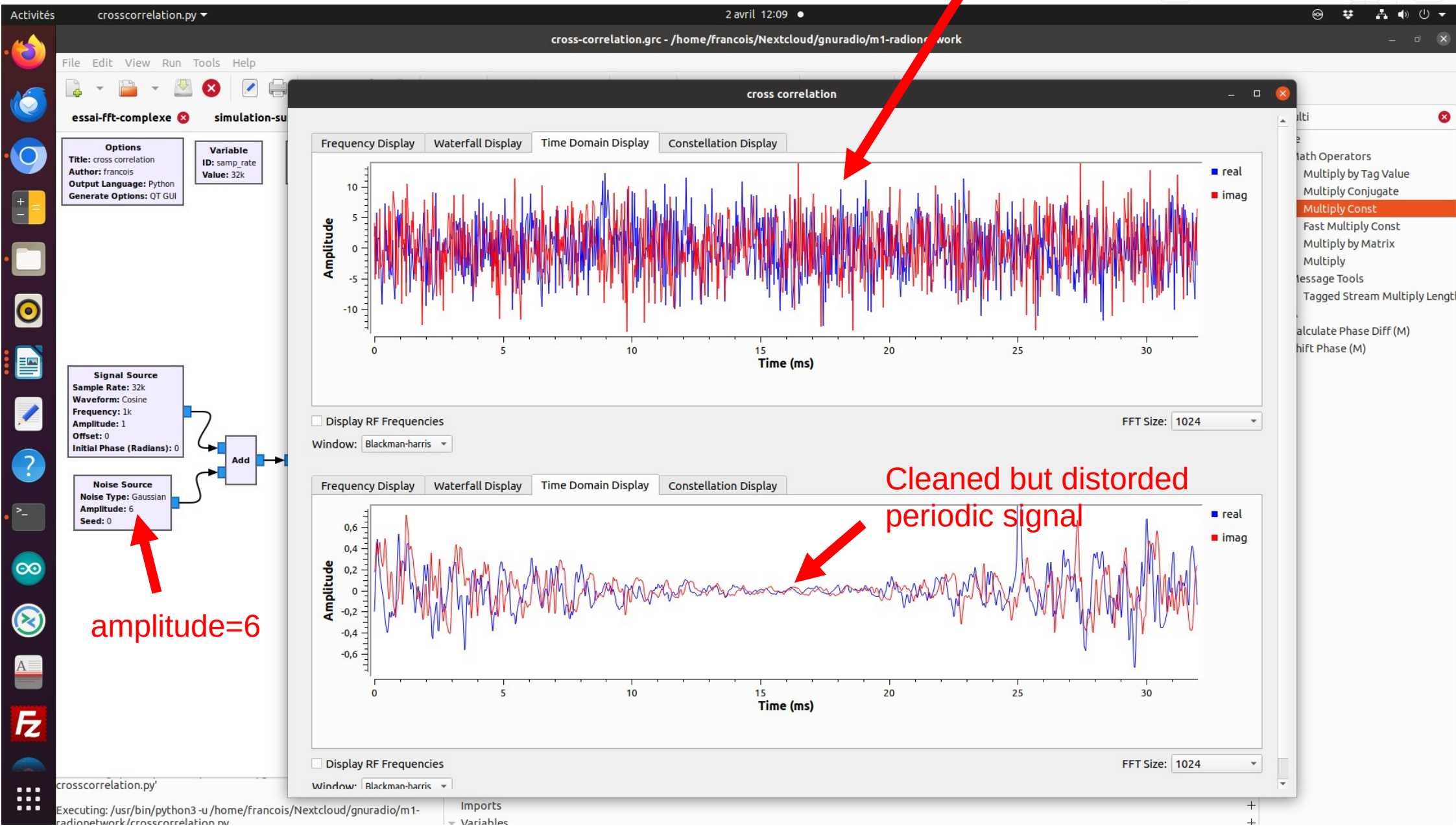


Crosscorrelation in GNU Radio

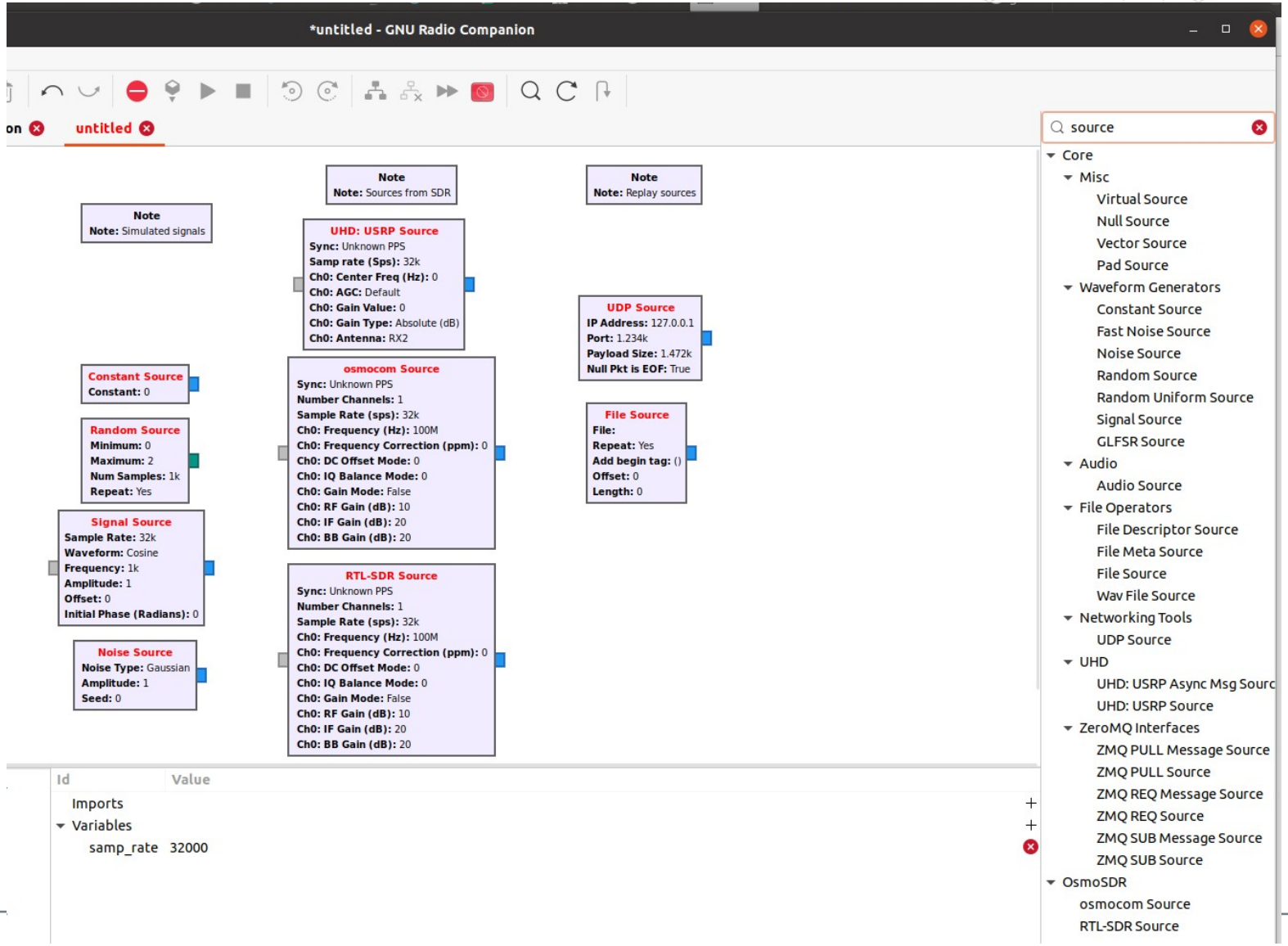
Orignal capture including signal and noise with amplitude=6



Crosscorrelation in GNU Radio

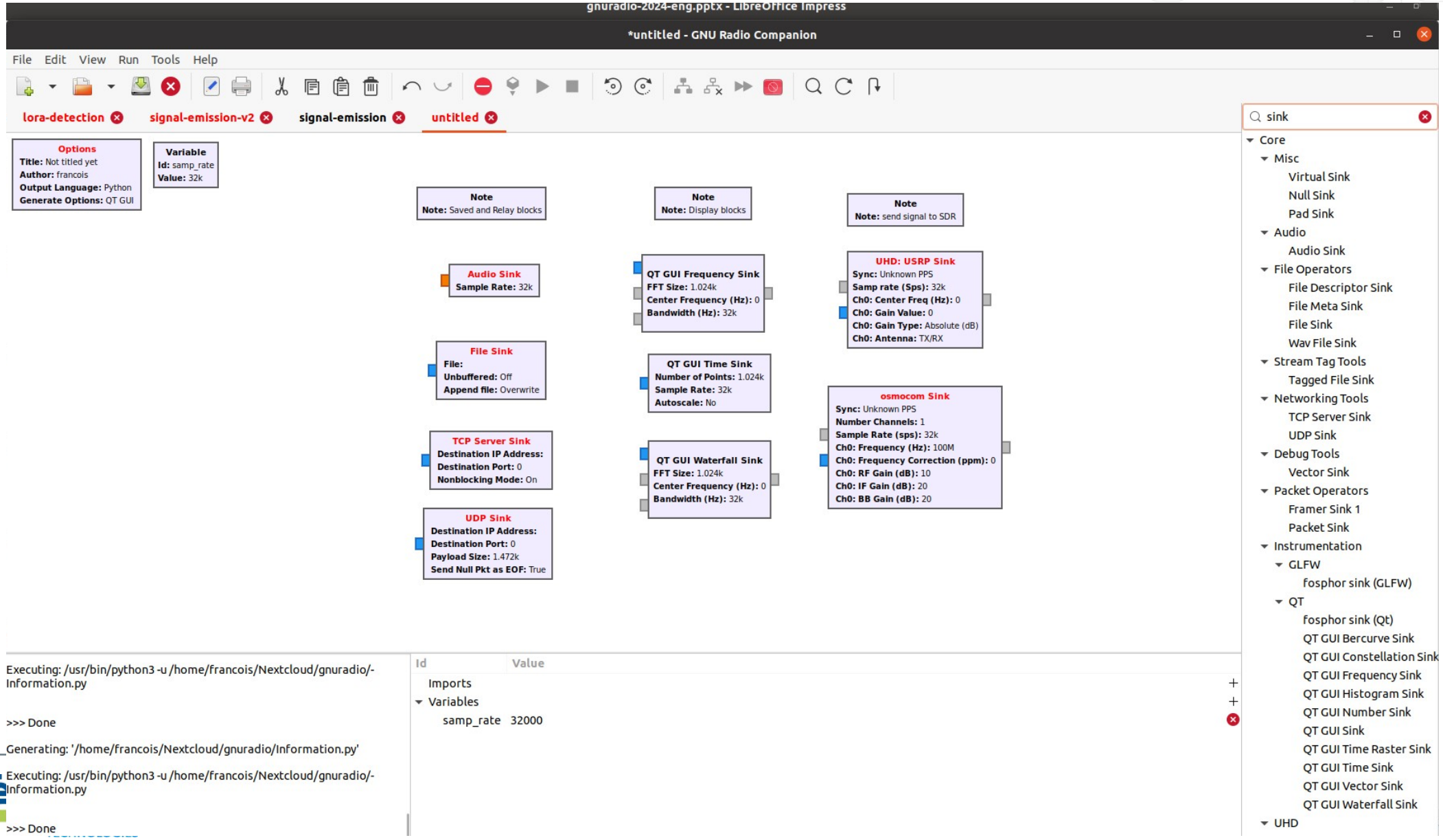


Few Source Blocks in GNU Radio



femto-st SCIENCES & TECHNOLOGIES

Few Sink Blocks in GNU Radio



Phase Interferometry

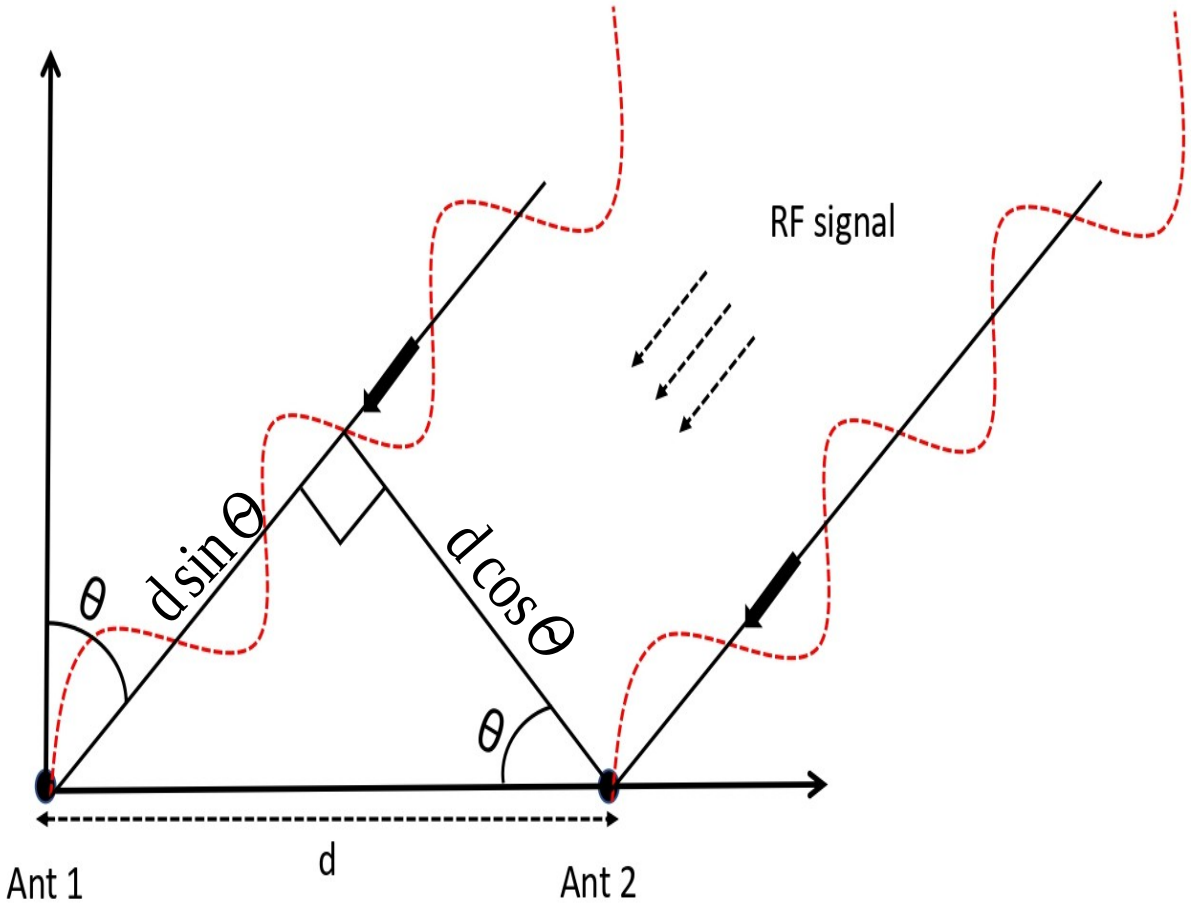
- Two antennas spaced by a distance *d* apart
- The phase difference between the two signals is :

$$\Delta\varphi$$

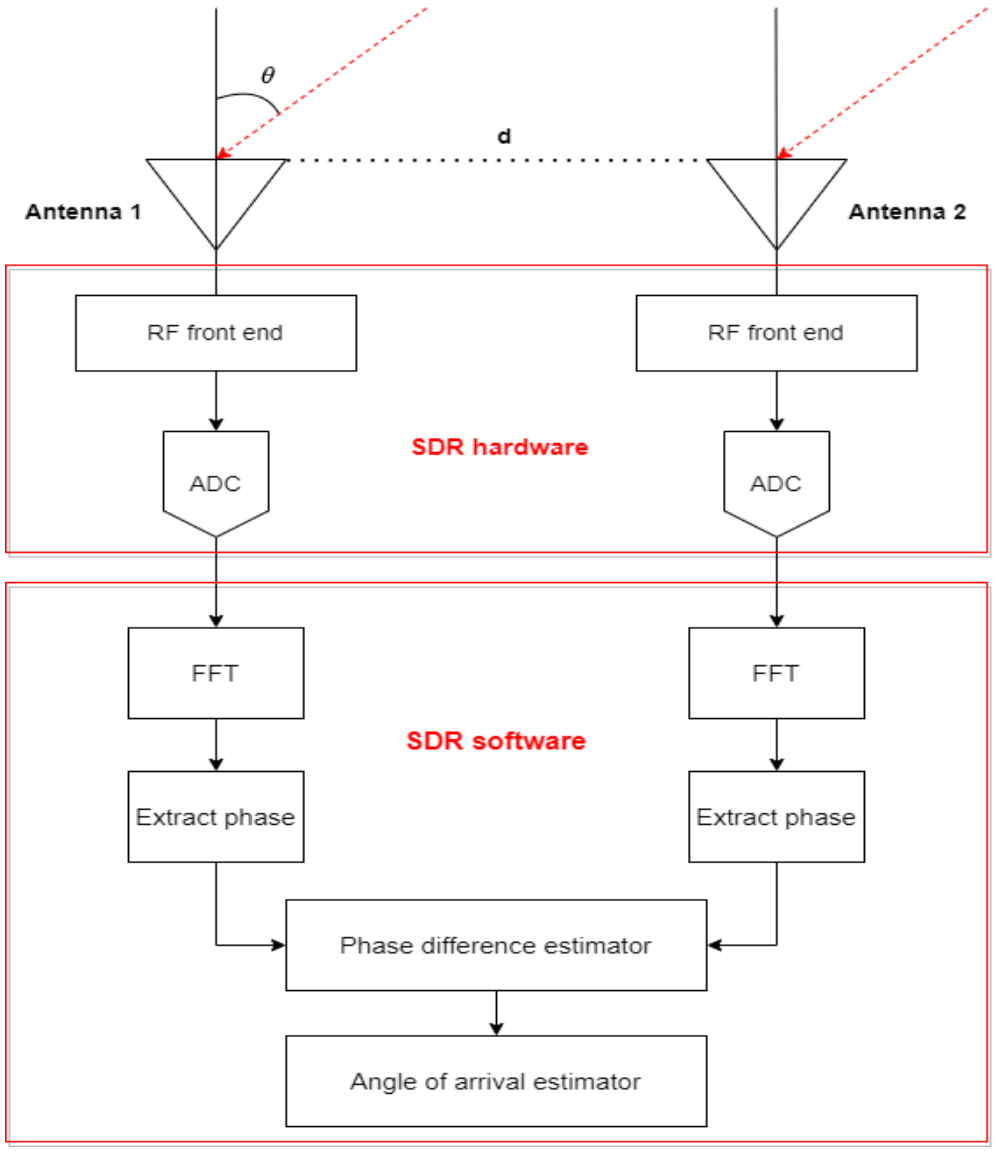
- The AoA (Angle of Arrival) of the user signal is:

$$\theta = \arccos\left(\frac{\Delta\varphi \,.\, \lambda}{2\pi \,.\, d}\right)$$

Where θ is the AoA, λ is the signal wavelength, $\Delta\varphi$ is the difference in phase between the two antennas, and *d* is the separation distance between the antennas

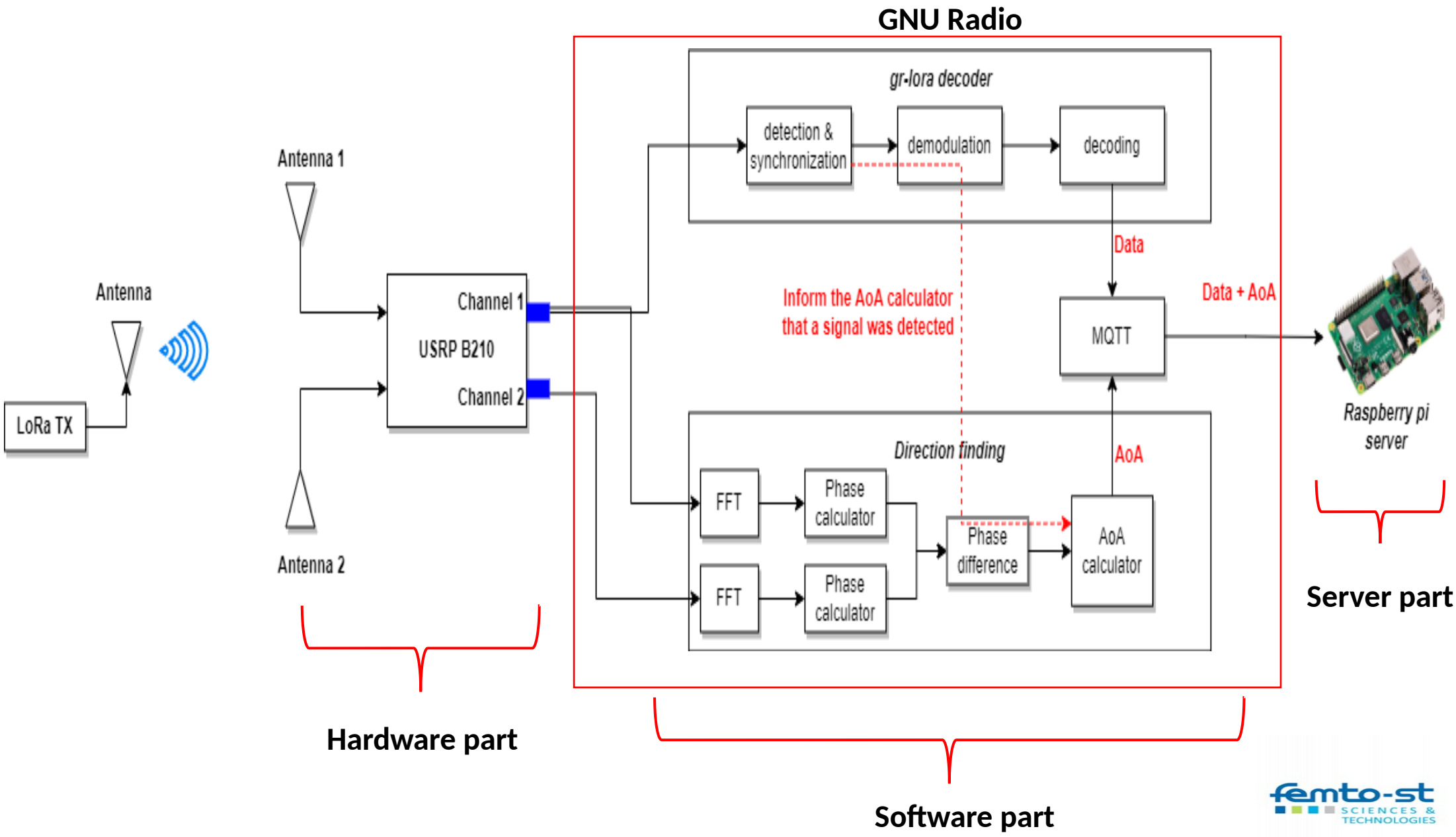


FFT Phase Interferometry

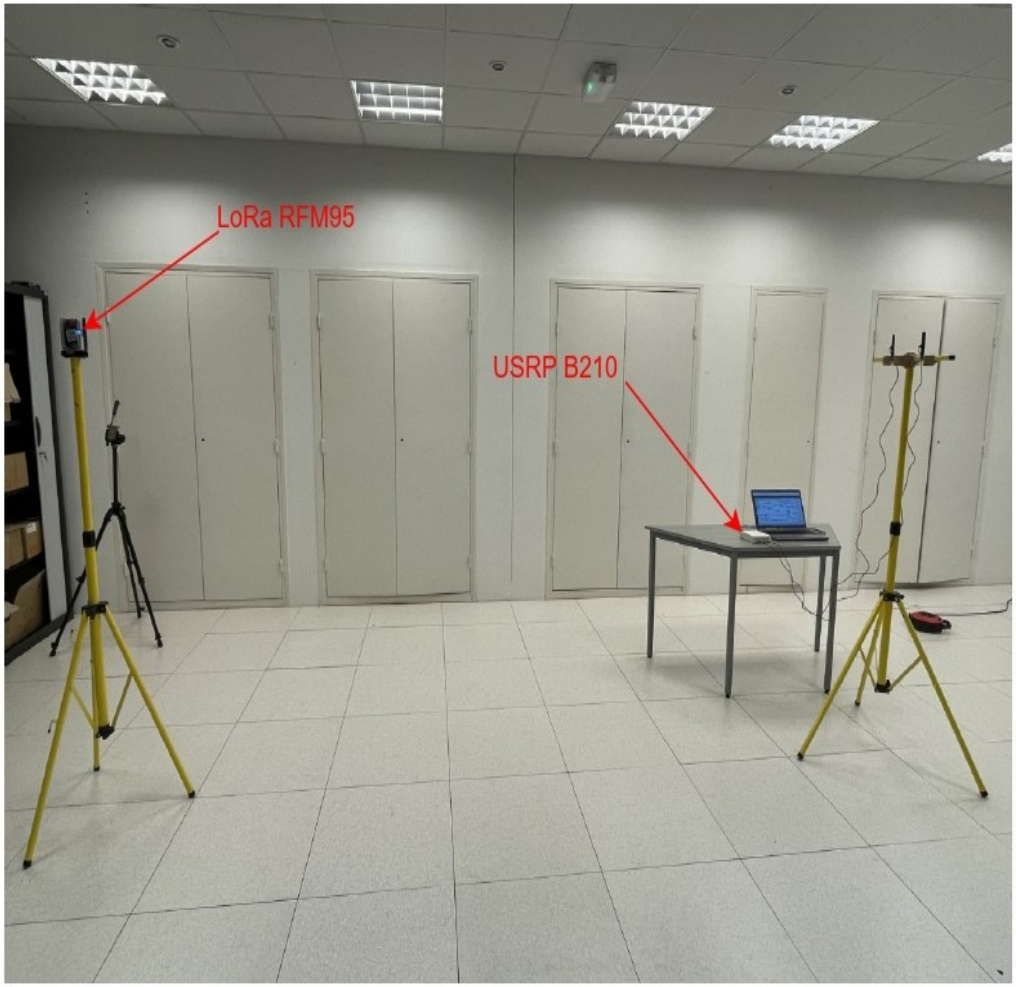


- The two antennas are spaced by a distance d apart
- FFT can implement a phase difference direction finding system
- Each output of the FFT consists of a real and imaginary term
- The phase of each filter FFT output is computed
- The AoA of the user signal

Signal Detector/AoA Calculator



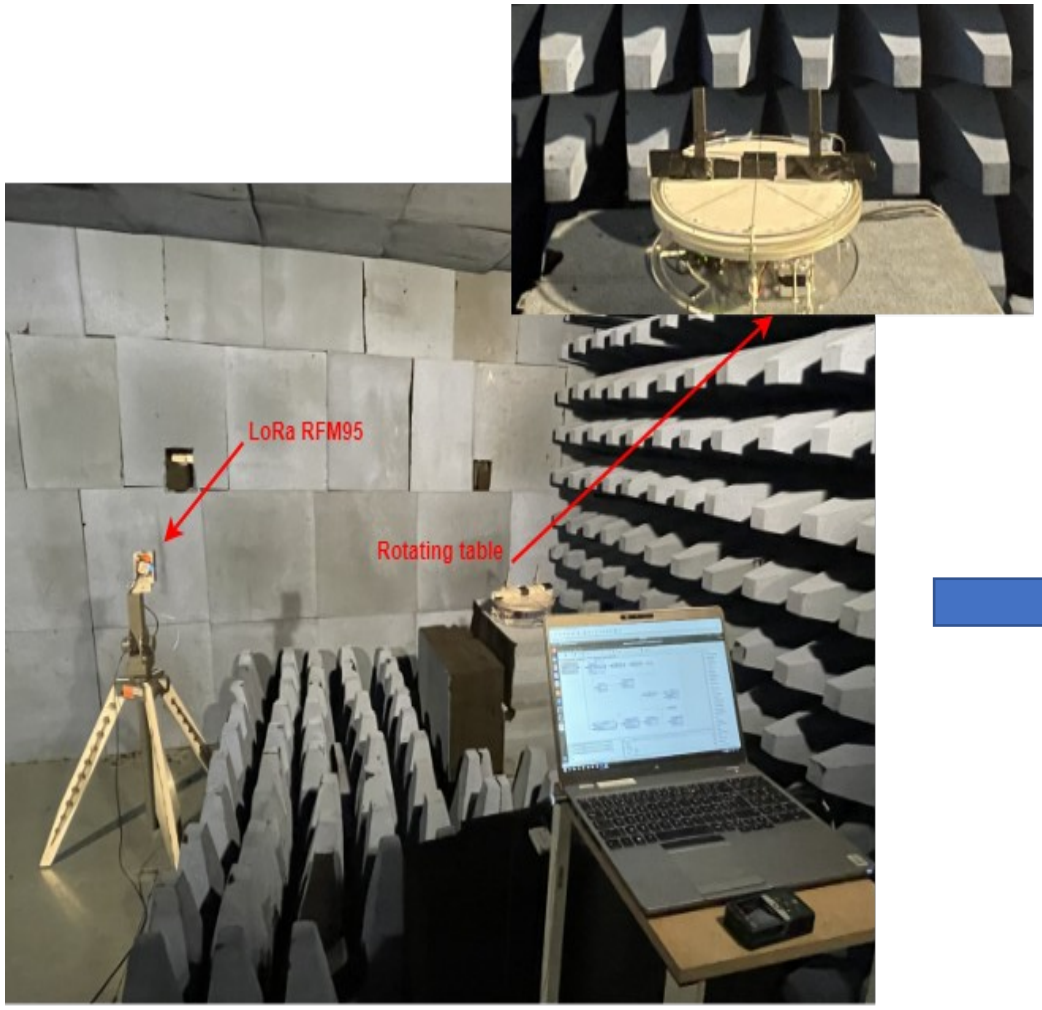
Experimental & results



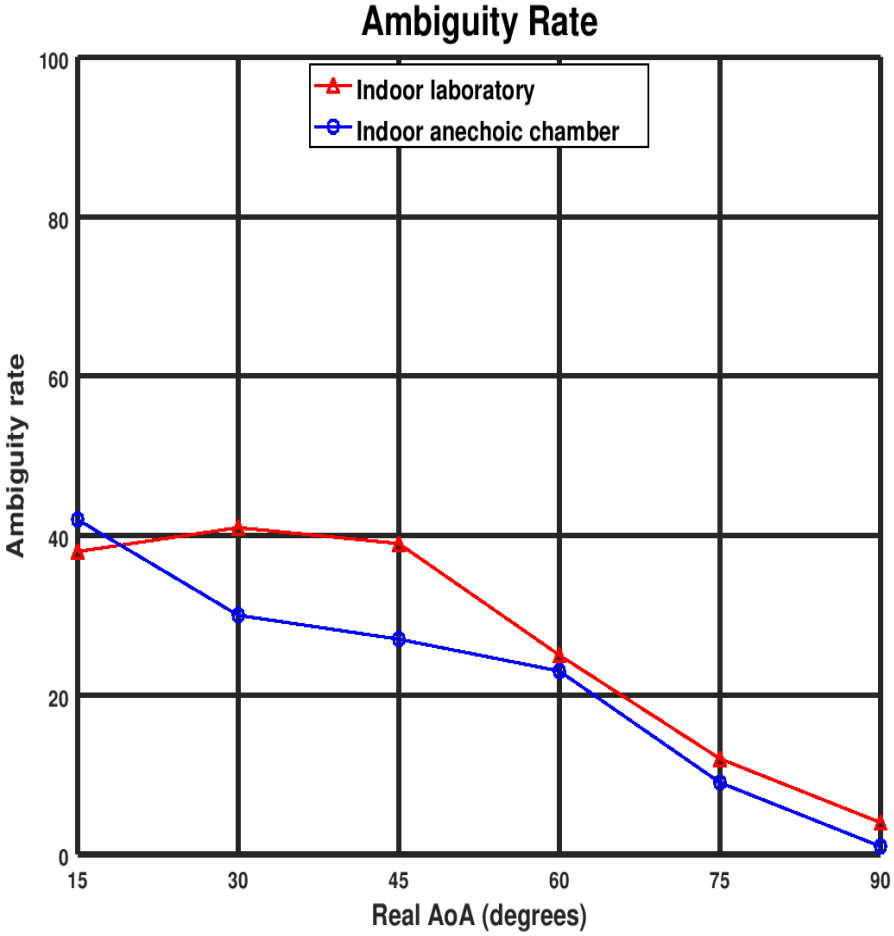
- Both devices were mounted on tripods with a height of ~1 meter
- Transmitter was configured to send LoRa signal with the following:
  - Carrier Frequency: 868 MHz
  - Spreading Factor: 7
  - Channel Bandwidth: 125 kHz
  - Coding Rate: 4/5
- The experiments were conducted into 2 parts:
  - Identifying the best antennas spacing
  - Measuring the AoAs in the range of [0,180] degrees interval

femto-st SCIENCES & TECHNOLOGIES

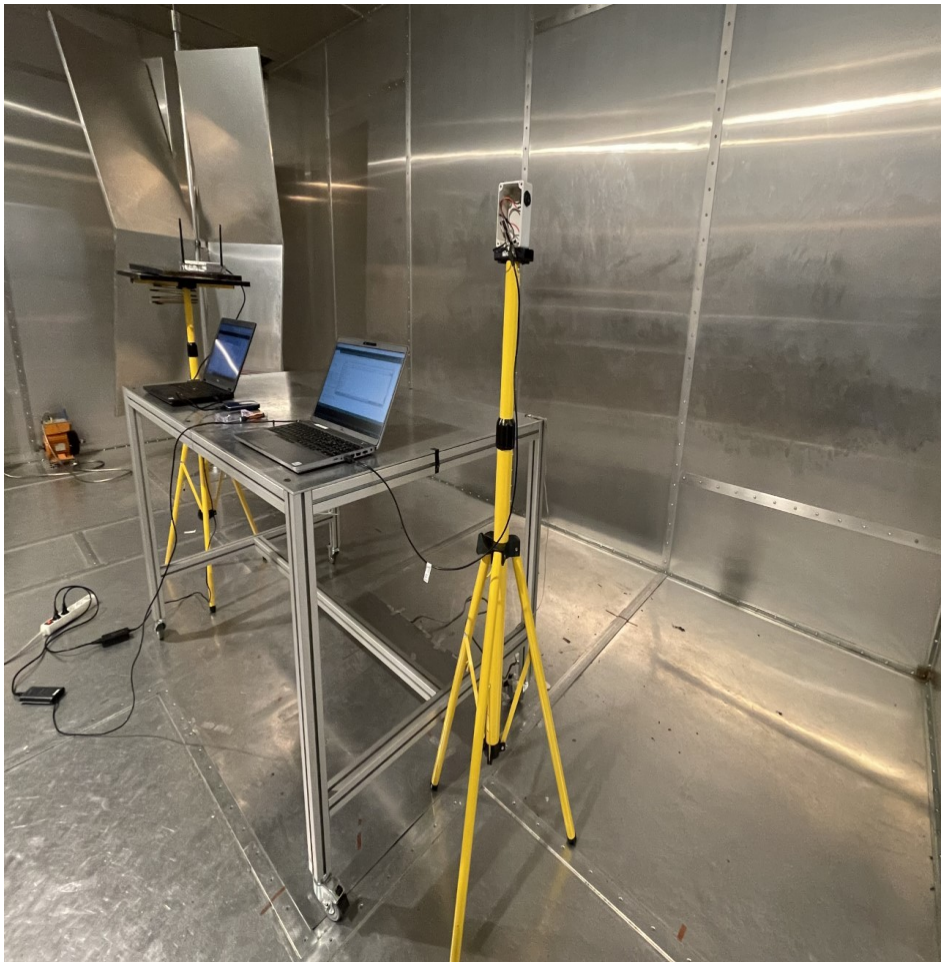
Anechoic room/chamber to avoid interferences



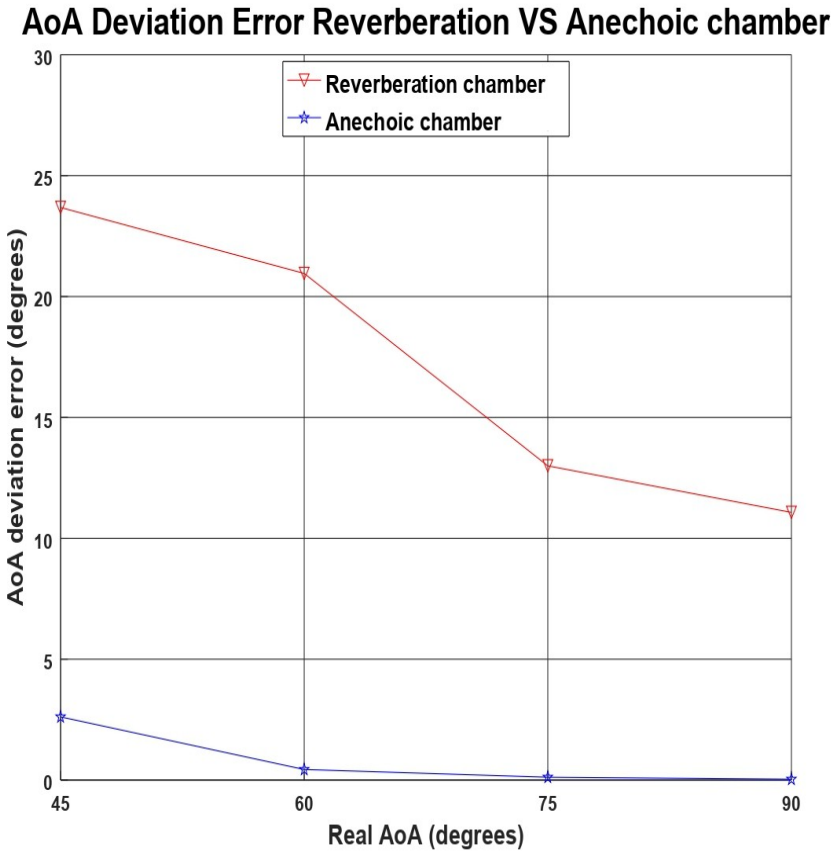
Test conducted at the IUT-Blagnac-Toulouse



Reverberation room



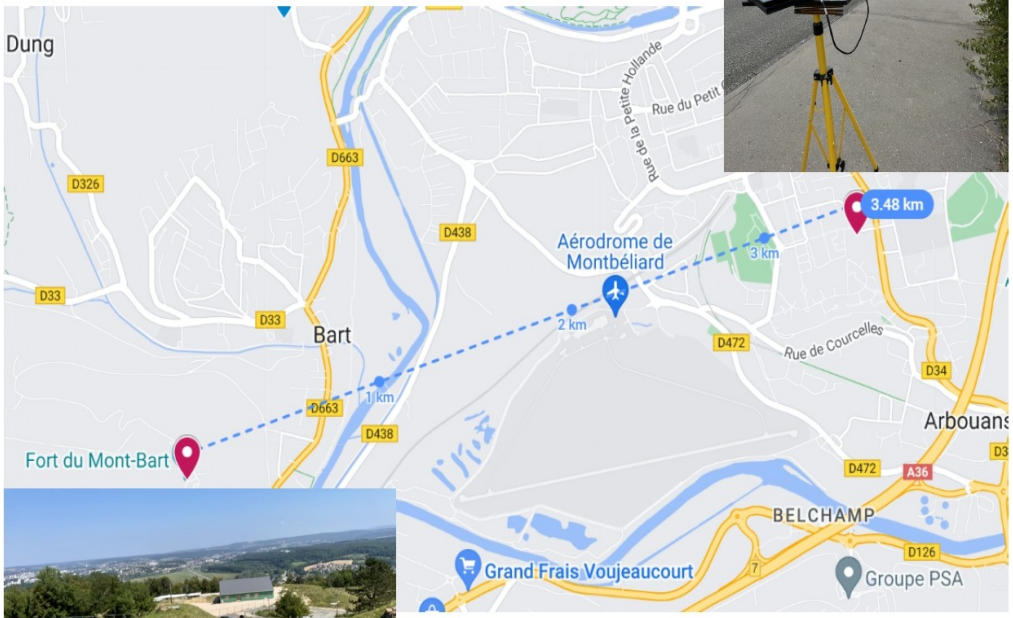
Test conducted at the UTBM (Belfort-Montbeliard)



- Each point corresponds to the mean of 30 AoA measurements
- For 90 degrees, the mean error was 12 degrees
- For 45 degrees, the mean error was 18 degres

Long-range communication (3.8 km)

Numerica - USRP B210 dual antenna receiver

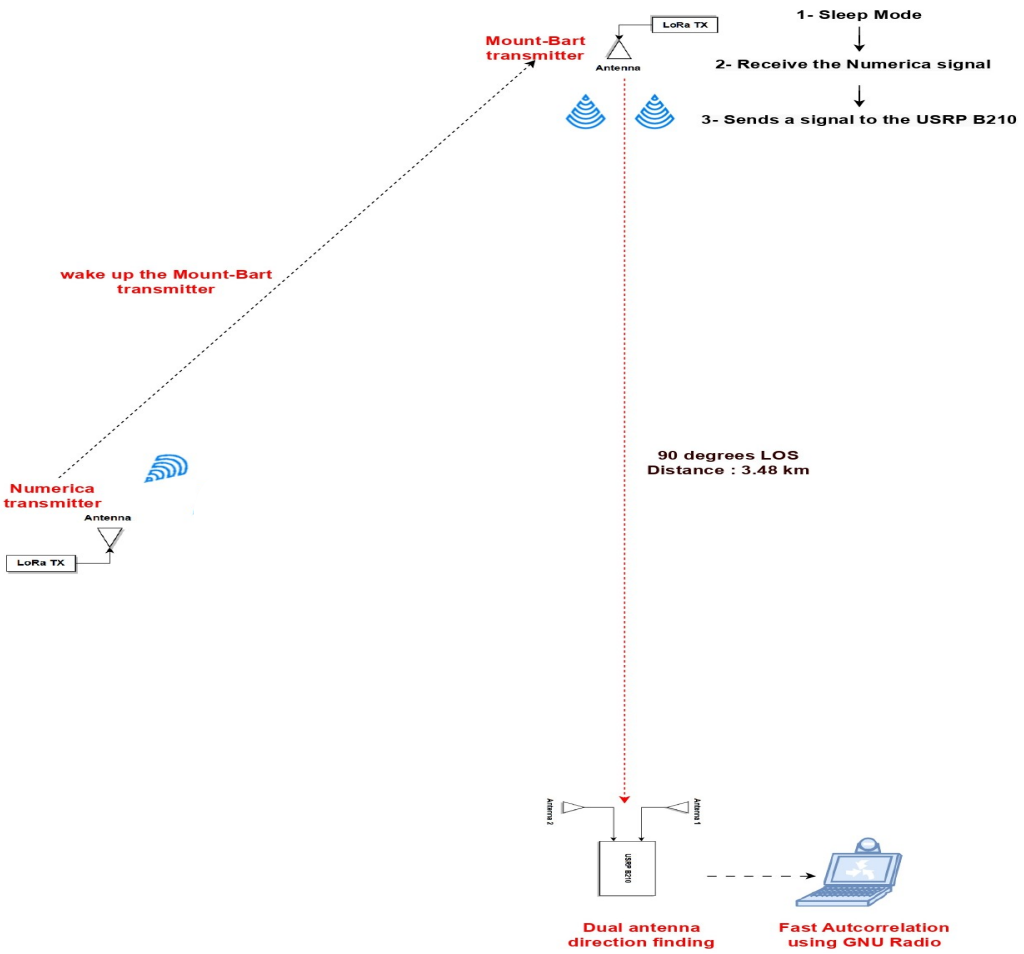


Mount-Bart LoRa transmitter

The Mount-Bart transmitter was located at a altitude 497 meters

The dual-antenna receiver was located at Numerica office

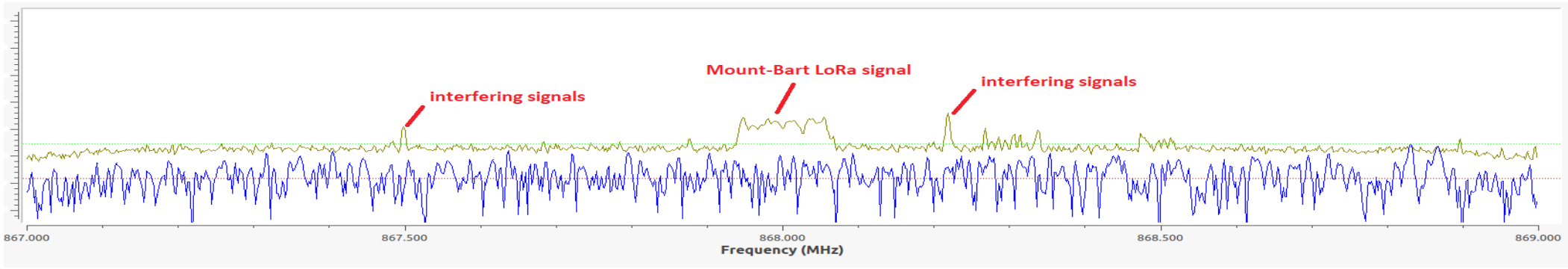
LOS conditions between the transmitter and the receiver



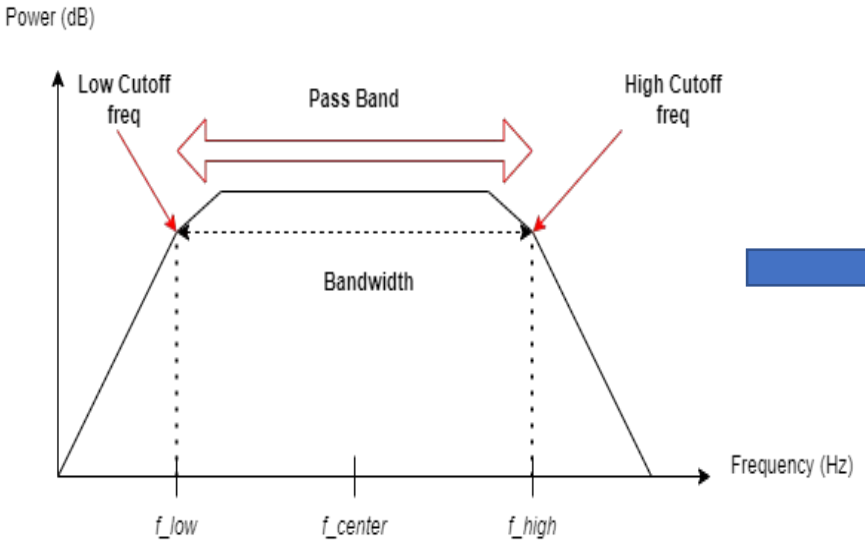
This setup save the battery lifetime of the transmitter while still enabling communication over long distance

The Mount-Bart transmitter was located at 90 degrees from the receiver

Long-range communication (3.8 km)



Removing interfering signals using a band pass filter

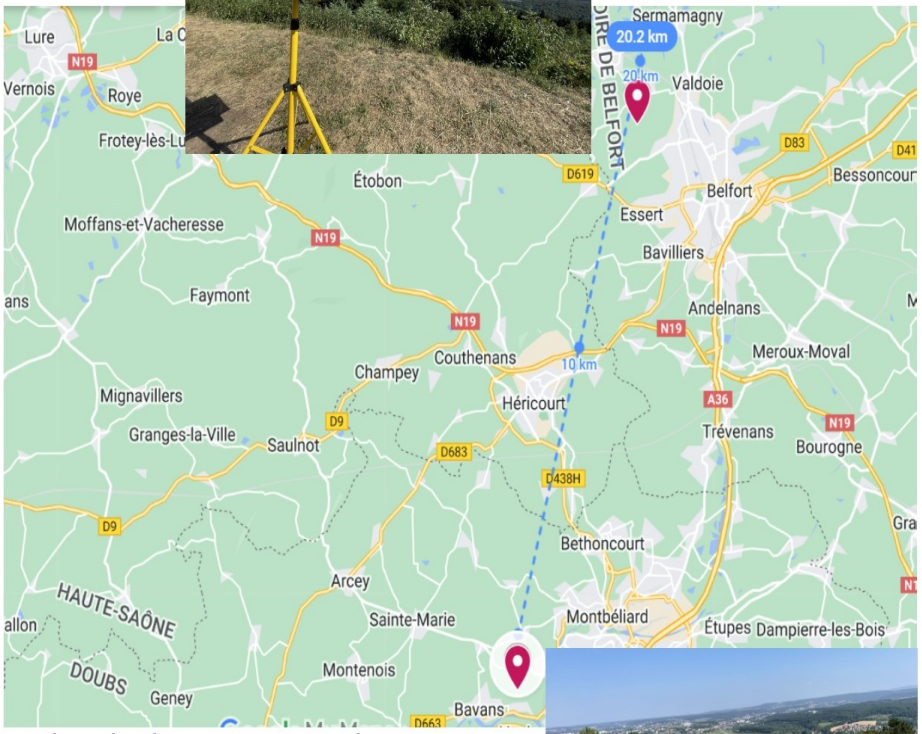


- **Baseband_center_freq = 0 Hz**
- **Bandwidth = 125 kHz**
- **Sampling_rate = 2 MHz**

- **Low Cutoff freq = -62.5 kHz**
- **High Cutoff freq = +62.5 kHz**

Long-range communication (20 km)

USRP B210 dual antenna receiver

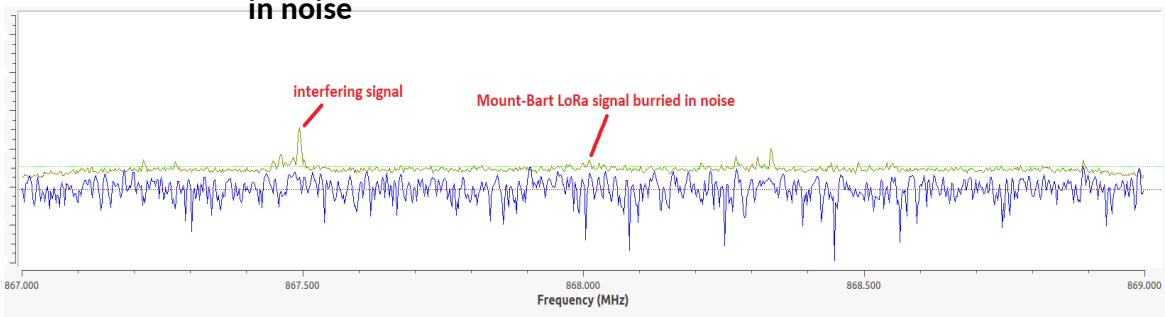


LoRa transmitter

The dual-antenna receiver was located at the mount of Salbert of the city of Belfort with an altitude of 650 m

The communication distance was 20.2 km in LoS conditions with 2 dBi antennas on both transmitter and receiver

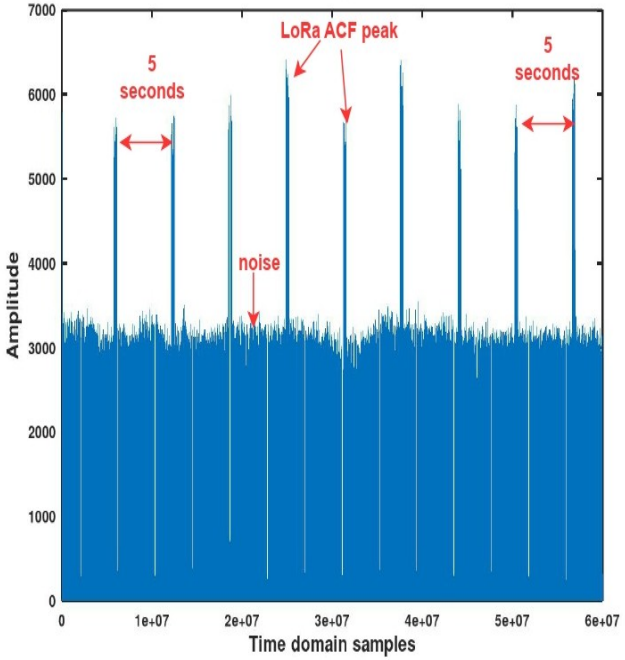
Frequency content of Mount Bart signal buried in noise



The Mount Bart signal can not be identified and detected

The Fast autocorrelation can be used to detect and identify the noisy Mount Bart signal

Autocorrelation of LoRa signals with negative SNR



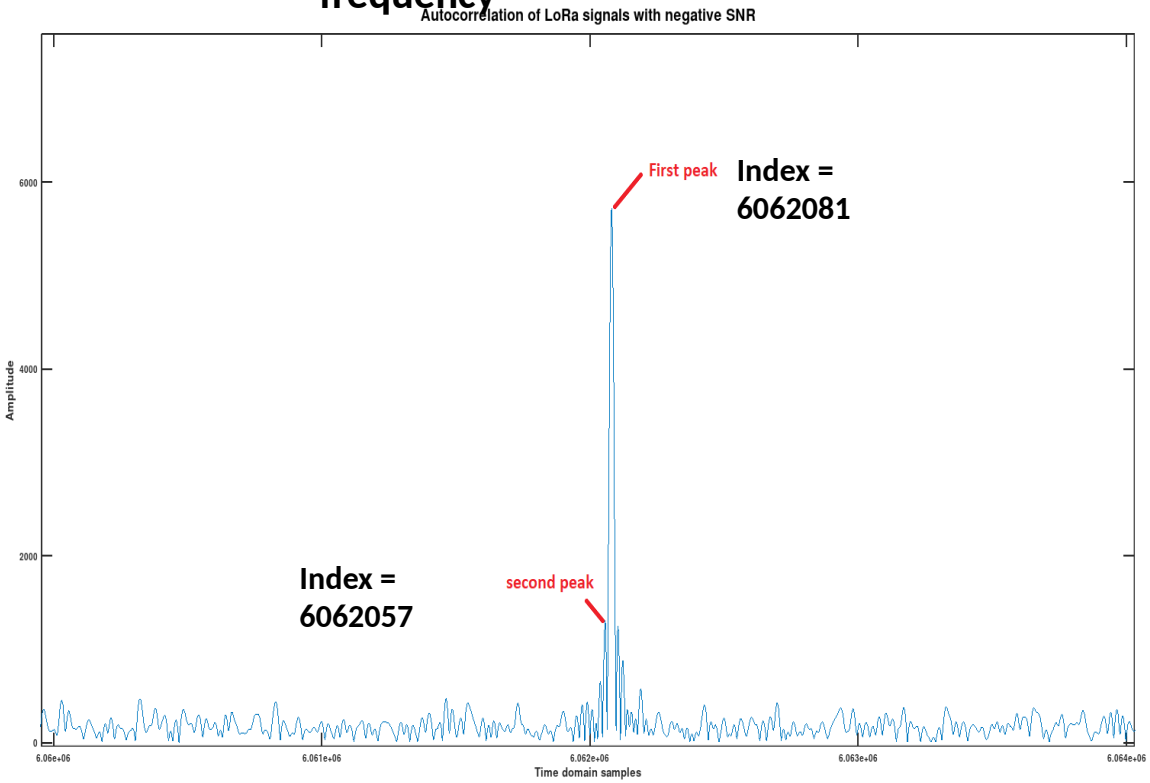
9 Mount bart LoRa signals was received and detected each 5 seconds

The ACF (Autocorrelation function) reveals 9 detections peaks

femto-st SCIENCES & TECHNOLOGIES

Long-range communication (20 km)

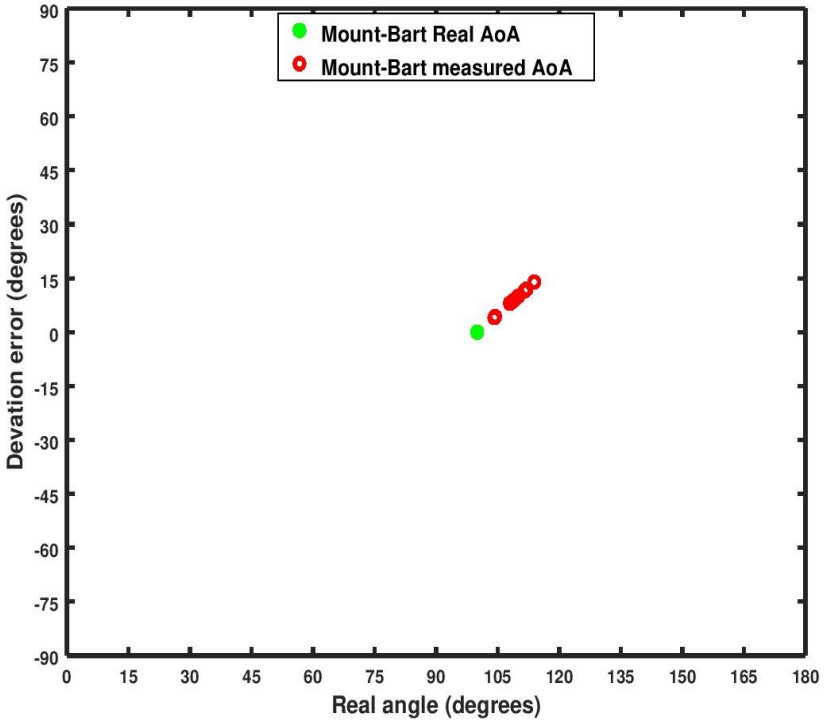
Identify the signal and its frequency



Carrier Frequency = 868 MHz + 83.3 kHz = 868.083 MHz

Frequency shift = 868.083 MHz – (868 MHz + 62.5 kHz) = 21 kHz

Measuring the AoAs

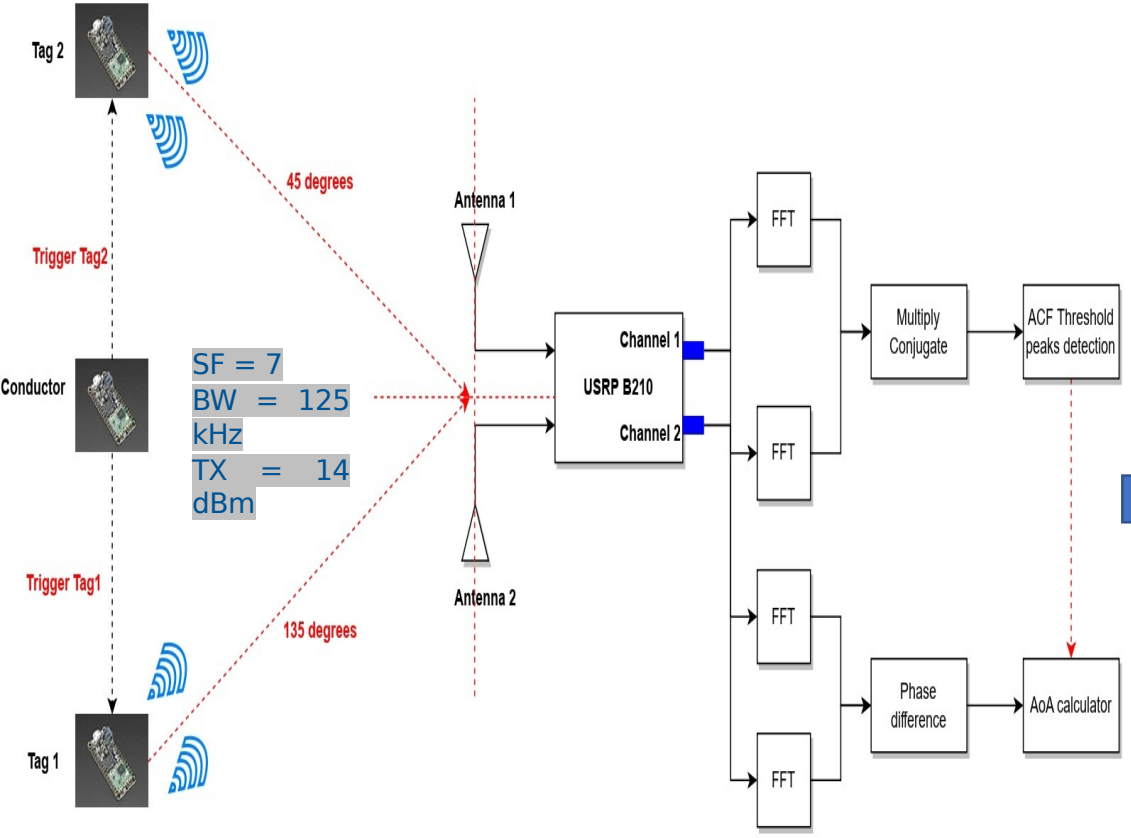


AoAs were measured for a rotation angle equals to 100 degrees

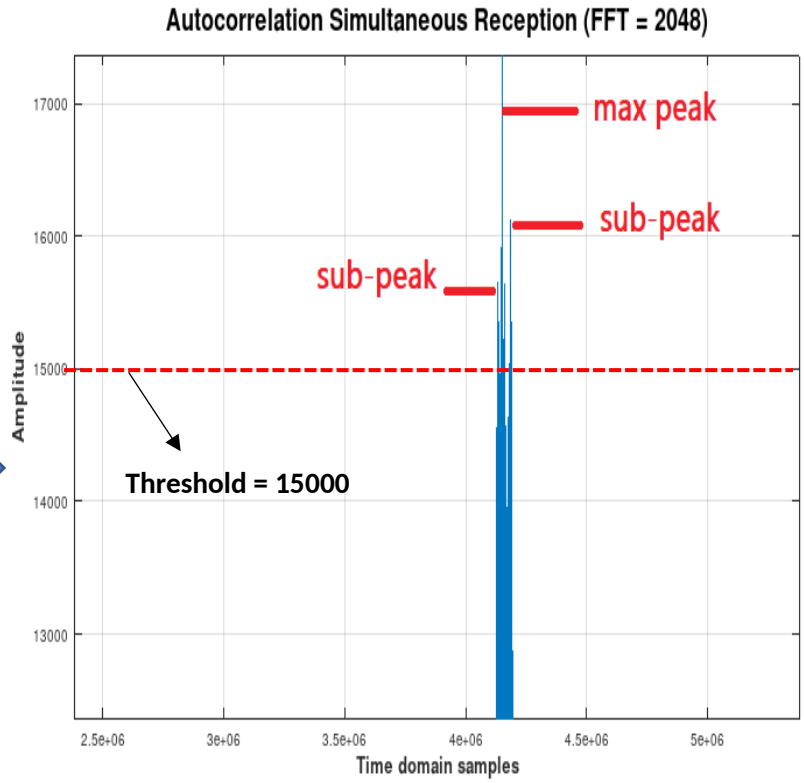
Measured AoAs ranged from 103 to 117 degrees

Managing interference

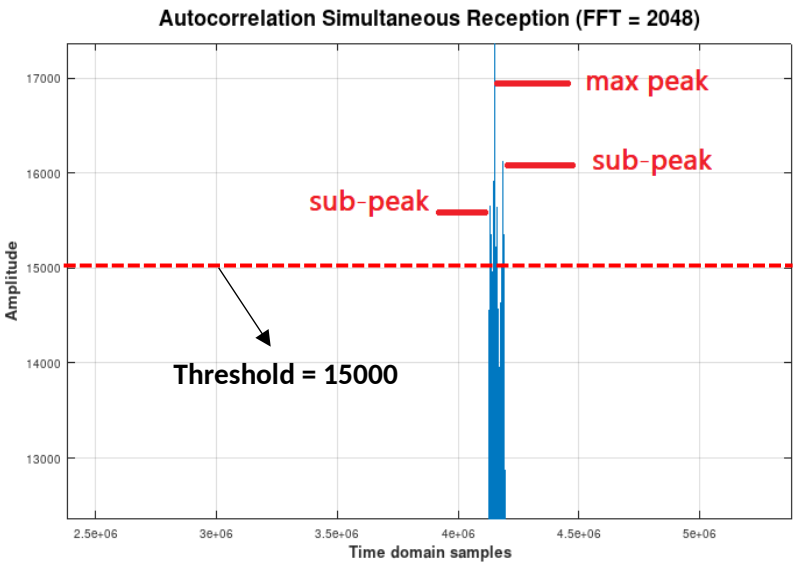
- With its CSS modulation and different SFs, LoRa allows orthogonal transmissions to avoid interferences
- Gr-LoRa is able to detect and decode two LoRa signals as long as they are on different SFs ; if else one signal will be lost
- Improving the decoding is not in our contribution scope -> The Fast ACF will be used to detect and measure AoAs of two concurrent signals



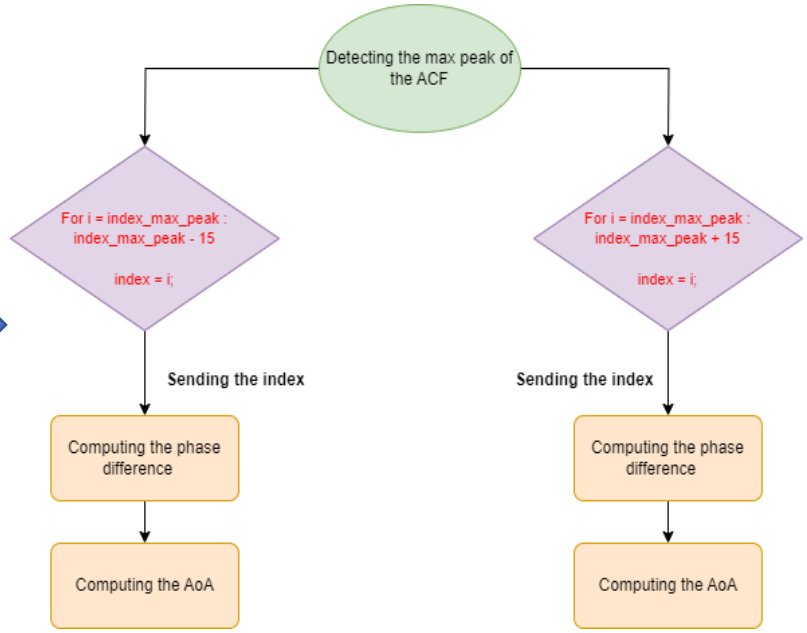
Result

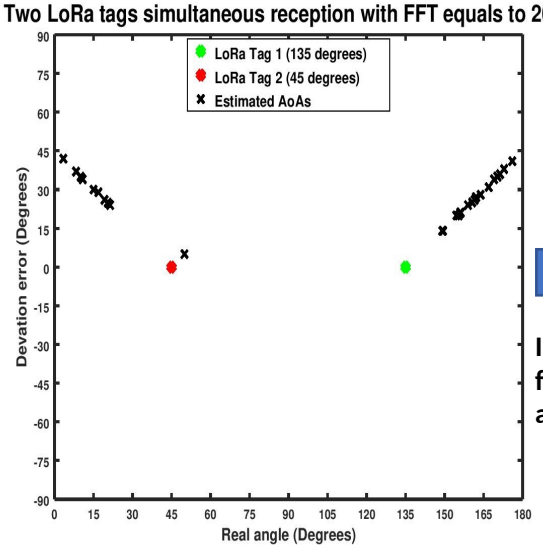


Interference / measuring AoAs

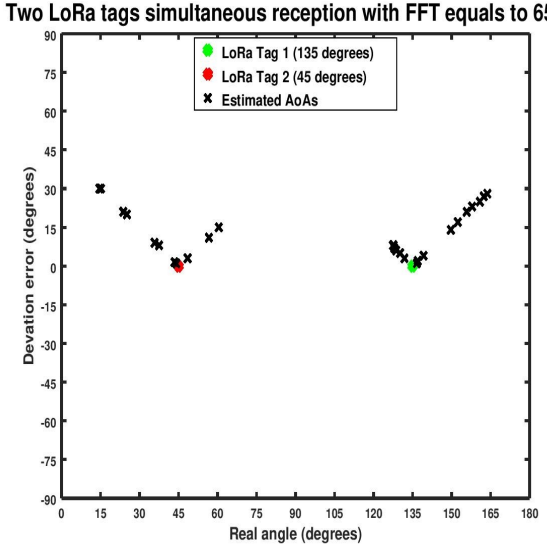


Proposed algorithm





Improving the frequency resolution and the AoAs



30 peaks were detected including a max peak **and 29 sub-peaks** around lag=0

Our algorithm was able to distinguish two distinct area of AoAs corresponding to **45 and 135 degrees**

It has been shown that increasing the FFT size increase the accuracy of our AoAs measurements

Increasing the FFT increase the frequency resolution more accurate detection of the max peak and sub-peaks

The **mean absolute error** of **transmitter 1 and 2** were **12 and 13 degrees** respectively

Compléments

- Faire un flot avec un filtre passe bas pour montrer. 1 freq basse et 1 freq haute, on filtre la haute et il reste la basse. On montre le résultat en temps et en fréquence
- Idem avec 3 freq pour le passe bande
- Montrer aussi de la cross-correlation (principe du GPS).
- Mélanger du bruit et du signal montrer en temps et en freq le résultat